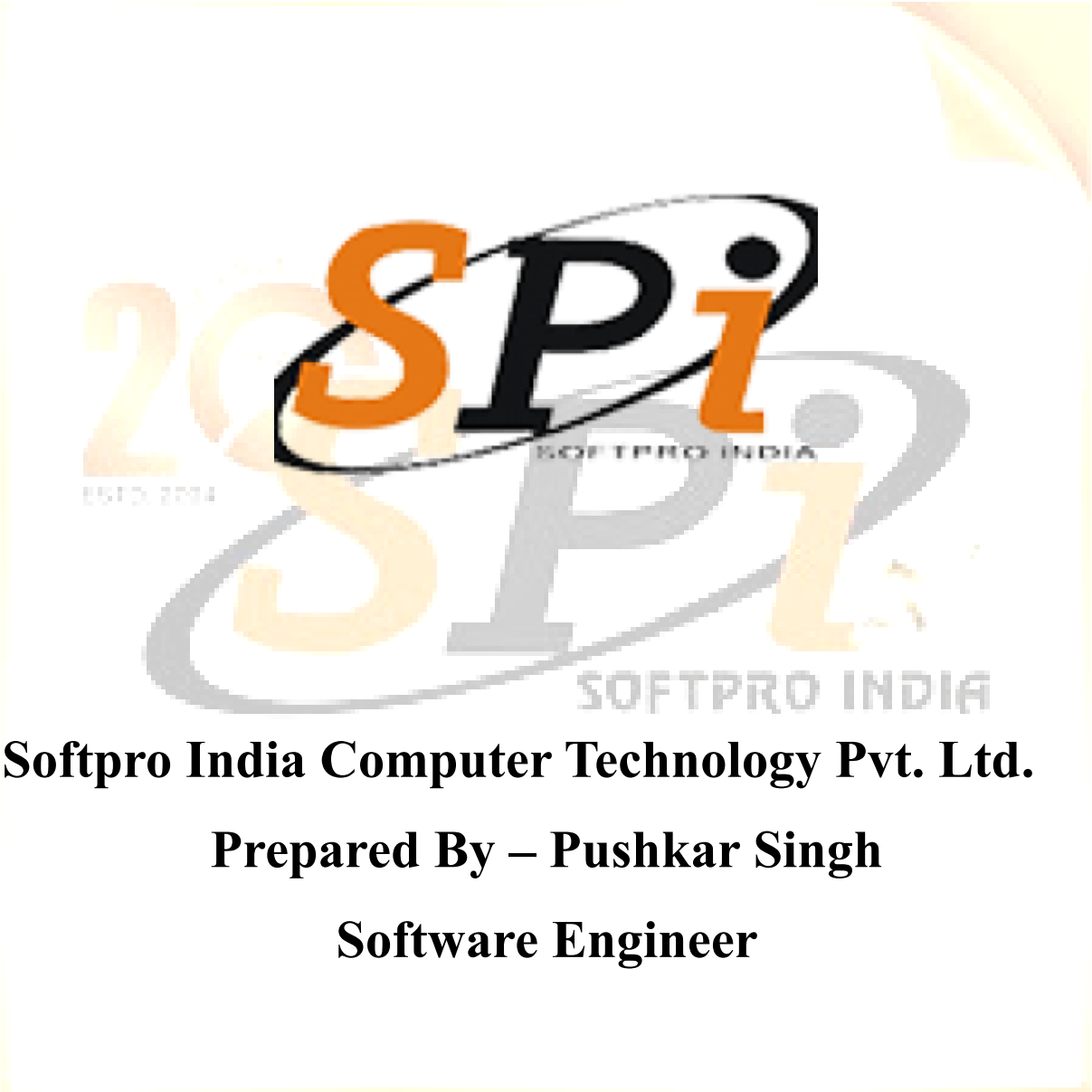
# MERN Stack Interview Question Booklet




**Softpro India Computer Technology Pvt. Ltd.**

**Prepared By – Pushkar Singh**

**Software Engineer**

# Topic 1: Database Fundamentals (MySQL & MongoDB Basics)

## Q1. What is a Database?

**Ans1.**
A **database** is an organized collection of data that can be easily accessed, managed, and updated. Databases store structured or unstructured information in a systematic way.

Examples:

- Customer information
- Product records
- Transaction details

**Hints:**

- Databases allow efficient **storage, retrieval, and manipulation** of data.

---

## Q2. What is DBMS?

**Ans2.**
A **Database Management System (DBMS)** is software that allows users to create, manage, and interact with databases.

Examples of DBMS:

- MySQL
- MongoDB
- PostgreSQL

Functions of DBMS:

- Data storage
- Data security
- Query processing
- Backup and recovery

**Hints:**
DBMS acts as an **interface between users and the database**.

---

## Q3. What is SQL?

**Ans3.**

SQL (Structured Query Language) is a language used to **interact with relational databases**.

Main uses:

- Creating databases
- Retrieving data
- Updating records
- Deleting records Example:

```
SELECT * FROM users;
```

**Hints:**
SQL works mainly with **tables, rows, and columns**.

---

## Q4. What is NoSQL?

**Ans4.**
NoSQL refers to **Non-Relational databases** designed for large-scale data storage and flexible schema.

Example:

- MongoDB

Characteristics:

- Schema-less design
- High scalability
- Stores JSON-like documents

**Hints:**
NoSQL databases are commonly used in **big data and real-time web applications**.

---

## Q5. What is the difference between SQL and NoSQL databases?

**Ans5.**

| Feature | SQL | NoSQL |
|---|---|---|
| Data Structure | Tables | Documents / Key-Value |
| Schema | Fixed | Flexible |
| Example | MySQL | MongoDB |
| Scaling | Vertical | Horizontal |

**Hints:**

- SQL is good for **structured data**
- NoSQL is good for **large, flexible data**

---

## Q6. What is MySQL?

**Ans6.**
MySQL is an **open-source relational database management system** that uses SQL for managing structured data.

Features:

- Table-based storage
- Strong ACID compliance
- Widely used with PHP and web applications

**Hints:**
Often used in the **LAMP stack (Linux, Apache, MySQL, PHP)**.

---

## Q7. What is MongoDB?

**Ans7.**
MongoDB is a **NoSQL document-oriented database** that stores data in **JSON-like BSON documents**.

Example Document:

```
{
  "name": "Pushkar",
  "age": 25,
  "skills": ["NodeJS", "MongoDB"]
}
```

**Hints:**
MongoDB stores data in **collections instead of tables**.

---

## Q8. What is a Table in MySQL?

**Ans8.**
A **table** is a structured format used to store data in rows and columns in relational databases like MySQL.

Example table:

| id | name | email |
|---|---|---|

1 Rahul rahul@mail.com

Hints:
Tables represent **entities** like users, orders, or products.

---

## Q9. What is a Collection in MongoDB?

**Ans9.**
A **collection** is a group of documents in MongoDB.

Example:

```
users collection
├── document1
├── document2
└── document3
```

Hints:
Collection in MongoDB ≈ **Table in MySQL**

---

## Q10. What is a Document in MongoDB?

**Ans10.**
A **document** is a single record stored inside a MongoDB collection.

Example:

```
{
  "_id": 1,
  "name": "Amit",
  "age": 24
}
```

Characteristics:

- Stored in BSON format
- Flexible schema
- Similar to JSON objects

Hints:
Document in MongoDB ≈ **Row in MySQL table**

**Topic 2: CRUD Operations (MySQL & MongoDB)**

**Q11. What is CRUD in databases?**

**Ans11.**

CRUD represents the four basic operations performed on database data:

- **Create** – Insert new data
- **Read** – Retrieve data
- **Update** – Modify existing data
- **Delete** – Remove data

These operations exist in both relational databases like MySQL and NoSQL databases like MongoDB.

**Hints:**
Every database application (login systems, blogs, ecommerce) uses CRUD operations.

---

**Q12. How do you create a database in MySQL?**

**Ans12.**
To create a database in MySQL, the **CREATE DATABASE** command is used.

Example:

```
CREATE DATABASE company_db;
```

To use the database:

```
USE company_db;
```

**Hints:**
Creating a database is the **first step before creating tables**.

---

**Q13. How do you create a table in MySQL?**

**Ans13.**
Tables are created using the **CREATE TABLE** statement.

Example:

```
CREATE TABLE users
( id INT PRIMARY
KEY,name
VARCHAR(50), email
VARCHAR(100)
);
```

Explanation:

- **id** – unique identifier

- **name** – user name
- **email** – user email

**Hints:**
Tables store structured data in rows and columns.

---

**Q14. How do you insert data in MySQL?**

**Ans14.**
The **INSERT INTO** statement is used to add records.

Example:

```
INSERT INTO users (id, name, email)
VALUES (1, 'Rahul', 'rahul@gmail.com');
```

Multiple records:

```
INSERT INTO users (id, name, email)
VALUES
(2, 'Aman', 'aman@gmail.com'),
(3, 'Ravi', 'ravi@gmail.com');
```

**Hints:**
INSERT is the **Create operation in CRUD**.

---

**Q15. How do you retrieve data from MySQL?**

**Ans15.**
The **SELECT** statement retrieves records from tables.

Example:

```
SELECT * FROM users;
```

Select specific columns:

```
SELECT name, email FROM users;
```

**Hints:**

- * means **all columns**
- SELECT is the **Read operation in CRUD**

---

**Q16. How do you update data in MySQL?**

**Ans16.**

The **UPDATE** command modifies existing records.

Example:

```
UPDATE users
SET email = 'rahul123@gmail.com'
WHERE id = 1;
```

Important:

- Always use **WHERE clause** to avoid updating all records.

**Hints:**
UPDATE represents the **Modify operation** in CRUD.

---

**Q17. How do you delete data in MySQL?**

**Ans17.**
The **DELETE** command removes records.

Example:

```
DELETE FROM users
WHERE id = 1;
```

Delete all rows:

```
DELETE FROM users;
```

**Hints:**
Use **WHERE clause** to avoid deleting the entire table accidentally.

---

**Q18. How do you insert a document in MongoDB?**

**Ans18.**
Documents are inserted using **insertOne()** or **insertMany()** in MongoDB.

Example:

```
db.users.insertOne(
{  name: "Rahul",
age: 25,  city:
"Delhi" });
```

Multiple documents:

```
db.users.insertMany([
{name:"Aman",age:22},
 {name:"Ravi",age:24}
]);
```

**Hints:**
Documents are stored in **collections**.

---

**Q19. How do you retrieve data from MongoDB?**

**Ans19.**
The **find()** method retrieves documents.

Example:

db.users.find();

Find specific

record:

db.users.find({name:"Rahul"});

**Hints:**

- find() returns **multiple documents**
- findOne() returns **one document**

---

**Q20. How do you update data in MongoDB?**

**Ans20.**
Data is updated using **updateOne()** or **updateMany()**.

Example:

```
db.users.updateOne(
{name:"Rahul"},
 {$set:{age:26}}
);
```

Explanation:

- **$set** updates specific fields.

**Hints:**
MongoDB uses **update operators** like:

- $set
- $inc
- $unset

# Topic 3: Keys, Constraints and Data Types (MySQL & MongoDB)

## Q21. What is a Primary Key in MySQL?

**Ans21.**
A **Primary Key** is a column (or combination of columns) that uniquely identifies each record in a table in MySQL.

Characteristics:

- Must be **unique**
- Cannot contain **NULL values**
- Only **one primary key** per table Example:

```
CREATE TABLE users (
id INT PRIMARY KEY,
 name VARCHAR(50)
);
```

Hints:
Primary Key ensures **data uniqueness**.

---

## Q22. What is a Foreign Key?

**Ans22.**
A **Foreign Key** is a column that creates a relationship between two tables.

It references the **Primary Key of another table**.

Example:

```
CREATE TABLE orders (
order_id INT,
 user_id INT,
 FOREIGN KEY (user_id) REFERENCES users(id)
);
```

Explanation:

- user_id connects **orders table** with **users table**

Hints:
Foreign keys help maintain **referential integrity**.

---

## Q23. What is a Unique Constraint?

**Ans23.**
A **UNIQUE constraint** ensures that all values in a column are different.

Example:

```
CREATE TABLE users (
id INT PRIMARY KEY,
 email VARCHAR(100) UNIQUE
);
```

Characteristics:

- No duplicate values allowed
- Multiple UNIQUE constraints can exist in a table

Hints:
Commonly used for **email, username, phone number**.

---

## Q24. What is NOT NULL constraint?

**Ans24.**
The **NOT NULL constraint** ensures that a column cannot have NULL values.

Example:

```
CREATE TABLE users (
id INT PRIMARY KEY,
 name VARCHAR(50) NOT NULL
);
```

Explanation:

- Every record must contain a **name value**.

Hints:
Used when a field is **mandatory**.

---

## Q25. What is AUTO_INCREMENT in MySQL?

**Ans25.**
AUTO_INCREMENT automatically generates a unique number whenever a new record is inserted.

Example:

```
CREATE TABLE users (
```

```
 id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(50)
);
```

Example insert:

```
INSERT INTO users (name) VALUES ('Rahul');
```

Hints:
IDs automatically become **1,2,3,4…**

---

## Q26. What is the _id field in MongoDB?

**Ans26.**
Every document in MongoDB contains a unique **_id field**.

Example:

```
{
 "_id": ObjectId("64ab12345"),
 "name": "Rahul",
 "age": 25
}
```

Characteristics:

- Automatically generated if not provided
- Works like **Primary Key**

Hints:
_id ensures every document is **unique**.

---

## Q27. What are MySQL Data Types?

**Ans27.**
Data types define the **type of data stored in a column**.

Common types in MySQL:

| Data Type | Description |
|---|---|
| INT | Integer numbers |
| VARCHAR | Variable-length string |
| TEXT | Long text |
| DATE | Date values |
| FLOAT | Decimal numbers |

Example:

```
CREATE TABLE products (
 id INT,
 name VARCHAR(100),
price FLOAT
);
```

Hints:
Choosing correct data types improves **performance and storage efficiency**.

---

## Q28. What are MongoDB Data Types?

**Ans28.**
MongoDB supports several BSON data types.

Common types:

- String
- Integer
- Boolean
- Date
- Array
- Object
- ObjectId Example:

```
{
 "name": "Amit",
 "age": 24,
 "skills": ["NodeJS", "MongoDB"]
}
```

Hints:
MongoDB allows **flexible schema**.

---

## Q29. What is an Index in a Database?

**Ans29.**
An **Index** improves the speed of data retrieval operations.

Instead of scanning the whole table, the database searches using an index.

Example in MySQL:

```
CREATE INDEX idx_name
ON users(name);
```

Hints:

- Index improves **read performance**
- Too many indexes can slow **insert/update**

---

### Q30. What is Indexing in MongoDB?

**Ans30.**
Indexing in MongoDB improves query performance.

Example:

```
db.users.createIndex({name:1})
```

Explanation:

- 1 means **ascending order**
- -1 means **descending order**

Hints:
MongoDB automatically creates an index on **_id field**.

# Topic 4: Joins and Relationships (MySQL & MongoDB)

### Q31. What is a JOIN in MySQL?

**Ans31.**
A **JOIN** is used to retrieve data from **multiple tables** based on a related column.

Example in MySQL:

```
SELECT users.name, orders.order_id
FROM users
JOIN orders
ON users.id = orders.user_id;
```

Explanation:

- users and orders tables are connected through user_id.

**Hints**:
JOIN helps combine **related data from different tables**.

---

### Q32. What are the different types of JOINs in MySQL?

**Ans32.**

Common JOIN types in MySQL:

1. **INNER JOIN** – Returns matching records from both tables
2. **LEFT JOIN** – Returns all records from left table
3. **RIGHT JOIN** – Returns all records from right table
4. **FULL JOIN** – Returns all records from both tables (not directly supported in MySQL)

**Hints**:
INNER JOIN is the **most commonly used join in interviews**.

---

## Q33. What is INNER JOIN?

**Ans33.**
**INNER JOIN** returns records where there is a match in both tables.

Example:

```
SELECT users.name, orders.order_id
FROM users
INNER JOIN orders ON
users.id = orders.user_id;
```

Explanation:

- Only users who have orders will appear.

**Hints**:
Think of INNER JOIN as **intersection of two tables**.

---

## Q34. What is LEFT JOIN?

**Ans34.**
**LEFT JOIN** returns all records from the left table and matching records from the right table.

Example:

```
SELECT users.name, orders.order_id
FROM users
LEFT JOIN orders
ON users.id = orders.user_id;
```

Explanation:

- Users without orders will show **NULL values**.

**Hints**:
LEFT JOIN ensures **all rows from left table appear**.

---

## Q35. What is RIGHT JOIN?

**Ans35.**
**RIGHT JOIN** returns all records from the right table and matching records from the left table.

Example:

```
SELECT users.name, orders.order_id
FROM users
RIGHT JOIN orders ON
users.id = orders.user_id;
```

Explanation:

- All orders will appear even if user data is missing.

**Hints**:
RIGHT JOIN is less commonly used compared to LEFT JOIN.

---

## Q36. What is a Self Join?

**Ans36.**
A **Self Join** joins a table with itself.

Example:

```
SELECT A.name, B.name AS manager
FROM employees A
JOIN employees B
ON A.manager_id = B.id;
```

Explanation:

- Used when a table has **hierarchical relationships**.

**Hints**:
Common example: **employee-manager relationship**.

---

## Q37. How are relationships handled in MongoDB?

**Ans37.**
Relationships in MongoDB are handled in two ways:

1. **Embedded Documents**
2. **Referenced Documents**

Unlike relational databases, MongoDB avoids complex joins.

**Hints**:
MongoDB focuses on **denormalized data models**.

---

## Q38. What are Embedded Documents in MongoDB?

**Ans38.**
Embedded documents store related data inside the same document.

Example:

```
{
 "name": "Rahul",
 "orders": [
  {"product": "Laptop", "price": 70000},
  {"product": "Mouse", "price": 500}
 ]
}
```

Advantages:

- Faster reads
- No joins required

**Hints**:
Best for **one-to-few relationships**.

---

## Q39. What are Referenced Documents in MongoDB?

**Ans39.**
Referenced documents store related data in separate collections and reference them using IDs.

Example:

Users collection:

```
{
 "_id": 1,
```

```
 "name": "Rahul"
}
```

Orders collection:

```
{
 "user_id": 1,
 "product": "Laptop"
}
```

**Hints**:
Used when **data grows large**.

---

### Q40. What is the Aggregation Framework in MongoDB?

**Ans40.**
The **Aggregation Framework** in MongoDB processes data records and returns computed results. Example:

```
db.orders.aggregate([
 { $match: { status: "completed" } },
 { $group: { _id: "$user_id", total: { $sum: "$amount" } } }
])
```

Explanation:

- $match filters data
- $group groups records

**Hints**:
Aggregation is similar to **GROUP BY in SQL**.

# Topic 5: Query Operations and Aggregation

### Q41. What is the GROUP BY clause in MySQL?

**Ans41.**
The **GROUP BY** clause groups rows that have the same values in specified columns into summary rows.

Example in MySQL:

```
SELECT department, COUNT(*)
```

```
FROM employees
GROUP BY department;
```

Explanation:

- Rows are grouped by the department column.
- COUNT(*) calculates the number of employees in each department.

**Hints**:
GROUP BY is often used with aggregate functions like **COUNT, SUM, AVG**.

---

## Q42. What is the HAVING clause?

**Ans42.**
The **HAVING** clause filters grouped results created by **GROUP BY**.

Example:

```
SELECT department, COUNT(*) AS total
FROM employees
GROUP BY department
HAVING total > 5;
```

Explanation:

- Only departments with more than 5 employees are returned.

**Hints**:
Difference:

- **WHERE** filters rows before grouping
- **HAVING** filters after grouping

---

## Q43. What is ORDER BY in MySQL?

**Ans43.**
The **ORDER BY** clause sorts query results in ascending or descending order.

Example:

```
SELECT * FROM users
ORDER BY name ASC;
```

Descending order:

```
SELECT * FROM users
ORDER BY age DESC;
```

**Hints**:

- ASC → Ascending (default)
- DESC → Descending

---

## Q44. What is LIMIT in MySQL?

**Ans44.**
The **LIMIT** clause restricts the number of rows returned by a query.

Example:

```
SELECT * FROM users
LIMIT 5;
```

Example with offset:

```
SELECT * FROM users
LIMIT 5 OFFSET 10;
```

Explanation:

- Skip first 10 records and return next 5.

**Hints**:
Commonly used for **pagination**.

---

## Q45. What are Aggregate Functions in MySQL?

**Ans45.**
Aggregate functions perform calculations on multiple rows and return a single result.

Common functions in MySQL:

| Function | Description |
|---|---|
| COUNT() | Counts rows |
| SUM() | Calculates total |
| AVG() | Calculates average |
| MAX() | Finds maximum value |
| MIN() | Finds minimum value |

Example:

```
SELECT AVG(salary) FROM employees;
```

**Hints**:

Used mainly with **GROUP BY**.

---

## Q46. How do you count documents in MongoDB?

**Ans46.**
Use the **countDocuments()** method in MongoDB.

Example:

db.users.countDocuments() With

condition:

db.users.countDocuments({age:25}

)

**Hints**:
Useful for **analytics and reporting queries**.

---

## Q47. What is the $match stage in MongoDB aggregation?

**Ans47.**
$match filters documents in the aggregation pipeline.

Example:

```
db.orders.aggregate([
 { $match: { status: "completed" } }
])
```

Explanation:

- Returns only orders with status **completed**.

**Hints**:
Similar to **WHERE clause in SQL**.

---

## Q48. What is the $group stage in MongoDB?

**Ans48.**
$group groups documents by a specified field and performs aggregation operations.

Example:

```
db.orders.aggregate([
 {
  $group: {    _id:
"$user_id",
totalOrders: { $sum: 1 }
  }
 }
])
```

Explanation:

- Groups orders by user_id.

**Hints**:
Equivalent to **GROUP BY in SQL**.

---

## Q49. What is the $sort stage in MongoDB?

**Ans49.**
$sort sorts documents in the aggregation pipeline.

Example:

```
db.users.aggregate([
{ $sort: { age: -1 } }
])
```

Explanation:

- -1 means descending order.

**Hints**:

- 1 → ascending
- -1 → descending

---

## Q50. What is the $limit stage in MongoDB?

**Ans50.**
$limit restricts the number of documents returned.

Example:

```
db.users.aggregate([
 { $limit: 5 }
])
```

Explanation:

- Returns only **5 documents**.

**Hints**:
Used for **pagination and performance optimization**.

# Topic 6: Normalization, Transactions, and Database Concepts

## Q51. What is Normalization in databases?

**Ans51.**
**Normalization** is the process of organizing data in a database to reduce **data redundancy** and improve data integrity.

It divides large tables into smaller related tables.

Used mainly in relational databases like MySQL.

Example:

Instead of storing repeated customer information in orders:

**Bad Table order_id customer_name**

**customer_email**

**Normalized Design**

Customers Table

| id | name | email |

Orders Table

| id | customer_id |

**Hints**:
Normalization helps maintain **consistent and clean data**.

---

## Q52. What are the different Normal Forms?

**Ans52.**

Common normal forms used in database design:

1. **1NF (First Normal Form)**
   - No repeating groups
   - Each column contains atomic values
2. **2NF (Second Normal Form)**
   - Must be in 1NF
   - No partial dependency
3. **3NF (Third Normal Form)**
   - Must be in 2NF
   - No transitive dependency

**Hints**:
Most databases are designed up to **3NF**.

---

## Q53. What is Denormalization?

**Ans53.**
**Denormalization** is the process of adding redundant data to improve **query performance**.

Example:

Instead of joining multiple tables, some data is duplicated.

Denormalization is commonly used in **NoSQL databases like** MongoDB.

**Hints**:
Trade-off:

- Faster reads
- More storage usage

---

## Q54. What are ACID properties?

**Ans54.**
ACID properties ensure reliable database transactions.

Supported strongly by MySQL.

ACID stands for:

1. **Atomicity** – Transaction happens completely or not at all
2. **Consistency** – Database remains valid after transaction
3. **Isolation** – Transactions do not interfere with each other
4. **Durability** – Changes remain even after system failure

**Hints**:
ACID ensures **data reliability and integrity**.

---

## Q55. What is a Transaction?

**Ans55.**
A **transaction** is a sequence of database operations executed as a single unit.

Example in MySQL:

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 100
WHERE id = 1;

UPDATE accounts
SET balance = balance + 100
WHERE id = 2;

COMMIT;
```

Explanation:

- Money transferred between accounts safely.

**Hints**:
If an error occurs → transaction can be **rolled back**.

---

## Q56. What is COMMIT in MySQL?

**Ans56.**
**COMMIT** saves all changes made during the transaction permanently.

Example:

```
COMMIT;
```

Explanation:

- Finalizes the transaction.

**Hints**:
After commit, changes **cannot be undone**.

---

## Q57. What is ROLLBACK?

**Ans57.**
**ROLLBACK** cancels a transaction and restores the database to its previous state.

Example:

```
ROLLBACK;
```

Explanation:

- If an error occurs, the database returns to its previous consistent state.

**Hints**:
Rollback is useful for **error handling**.

---

## Q58. What is the CAP Theorem?

**Ans58.**
The **CAP theorem** states that a distributed database system can guarantee only two of the following three properties:

1. **Consistency** – Every read receives the latest data
2. **Availability** – Every request receives a response
3. **Partition Tolerance** – System continues working despite network failures

Systems like MongoDB often prioritize **Availability and Partition tolerance**.

**Hints**:
CAP theorem is important in **distributed databases**.

---

## Q59. Does MongoDB support Transactions?

**Ans59.**
Yes, modern versions of MongoDB support **multi-document transactions**.

Example:

```
session.startTransaction()

db.orders.insertOne({...})

db.inventory.updateOne({...})

session.commitTransaction()
```

**Hints**:
Earlier MongoDB supported only **single-document atomic operations**.

---

### Q60. What is Atomic Operation in MongoDB?

**Ans60.**
An **atomic operation** means the operation completes fully or not at all.

In MongoDB, updates on a **single document** are atomic.

Example:

```
db.users.updateOne(
{name:"Rahul"},
 {$inc:{age:1}}
)
```

**Hints**:
Atomic operations ensure **data consistency during updates**.

# Topic 7: Indexing and Query Optimization

### Q61. What is Database Indexing?

**Ans61.**
**Indexing** is a technique used to improve the speed of data retrieval operations in a database.

Instead of scanning the entire table, the database uses an index to locate data quickly.

Example in MySQL:

```
CREATE INDEX idx_name
ON users(name);
```

**Hints**:

- Improves **SELECT query performance**
- Uses additional storage

---

### Q62. What are the types of indexes in MySQL?

**Ans62.**

Common index types in MySQL:

1. **Primary Index**

2. **Unique Index**
3. **Composite Index**
4. **Full-text Index**
5. **Spatial Index**

**Hints**:
Indexes help optimize **search operations**.

---

## Q63. What is a Composite Index?

**Ans63.**
A **Composite Index** is an index created on **multiple columns**.

Example:

```
CREATE INDEX idx_user_email
ON users(name, email);
```

Explanation:
The database can search faster when queries involve **both columns**.

**Hints**:
Order of columns in composite indexes **matters**.

---

## Q64. What is a Clustered Index?

**Ans64.**
A **Clustered Index** determines the physical order of data in a table.

In MySQL (InnoDB engine):

- The **Primary Key automatically becomes the clustered index**.

Characteristics:

- Only **one clustered index per table**
- Data stored in sorted order

**Hints**:
Clustered indexes improve **range queries performance**.

---

## Q65. What is a Non-Clustered Index?

**Ans65.**
A **Non-Clustered Index** stores a separate structure that points to the actual data rows.

Example:

```
CREATE INDEX idx_email
ON users(email);
```

Characteristics:

- Multiple non-clustered indexes can exist
- Improves search performance

**Hints**:
Used for **frequently searched columns**.

---

## Q66. What is Query Optimization?

**Ans66.**
**Query optimization** is the process of improving database query performance.

Common techniques:

- Creating indexes
- Avoiding unnecessary joins
- Selecting only required columns
- Using proper WHERE conditions Used in relational databases like MySQL.

**Hints**:
Efficient queries reduce **CPU usage and response time**.

---

## Q67. What is the EXPLAIN statement in MySQL?

**Ans67.**
The **EXPLAIN** statement shows how the database executes a query.

Example:

```
EXPLAIN SELECT * FROM users WHERE email='test@gmail.com';
```

Explanation:
It displays information such as:

Which index is used
Query execution plan

Number of rows scanned

**Hints**:
EXPLAIN helps **optimize slow queries**.

---

## Q68. What is Indexing in MongoDB?

**Ans68.**
Indexing in MongoDB improves query performance by allowing faster search of documents.

Example:

```
db.users.createIndex({email:1})
```

Explanation:

→ ascending index

**Hints**:
MongoDB automatically creates an index on the **_id field**.

---

## Q69. What is a Compound Index in MongoDB?

**Ans69.**
A **Compound Index** is an index created on multiple fields.

Example:

```
db.users.createIndex({name:1, age:-1})
```

Explanation:

- name sorted ascending
- age sorted descending

**Hints**:
Useful for queries that filter by **multiple fields**.

---

## Q70. What is a Covered Query in MongoDB?

**Ans70.**
A **Covered Query** is a query where all required fields are retrieved from the index itself without accessing the actual documents.

Example:

```
db.users.find(
{name:"Rahul"},
 {name:1,_id:0}
)
```

If an index exists on name, MongoDB can return results directly from the index.

**Hints**:
Covered queries are **extremely fast**.

# Topic 8: Views, Stored Procedures, Triggers, and MongoDB Collections

## Q71. What is a View in MySQL?

**Ans71.**
A **View** is a virtual table created from the result of a SQL query. It does not store data itself; it shows data stored in underlying tables.

Example in MySQL:

```
CREATE VIEW active_users AS
SELECT id, name, email
FROM users
WHERE status = 'active';
```

Explanation:

- The view behaves like a table.
- It simplifies complex queries.

**Hints**:
Views improve **security and code readability**.

---

## Q72. What are the advantages of Views?

**Ans72.**

Advantages in MySQL:

- Simplifies complex queries
- Improves security by hiding sensitive columns
- Provides logical data abstraction
- Helps reuse query logic

Example use case:

Showing only **public user data** without exposing passwords.

**Hints**:
Views help **control access to data**.

---

## Q73. What is a Stored Procedure?

**Ans73.**
A **Stored Procedure** is a precompiled collection of SQL statements stored in the database.

Example:

```
DELIMITER //

CREATE PROCEDURE GetUsers()
BEGIN
 SELECT * FROM users;
END //

DELIMITER ;
```

Execution:

```
CALL GetUsers();
```

**Hints**:
Stored procedures improve:

- Performance
- Code reuse
- Security

---

## Q74. What is a Trigger in MySQL?

**Ans74.**
A **Trigger** is a database object that automatically executes when a specified event occurs on a table.

Events:

- INSERT
- UPDATE
- DELETE

Example in MySQL:

```
CREATE TRIGGER before_user_insert
BEFORE INSERT ON users
FOR EACH ROW
SET NEW.created_at = NOW();
```

Explanation:

- Automatically sets a timestamp before inserting a record.

**Hints**:
Triggers automate **data validation and logging**.

---

## Q75. What is a Capped Collection in MongoDB?

**Ans75.**
A **Capped Collection** is a fixed-size collection in MongoDB.

Characteristics:

- Automatically overwrites oldest documents
- Maintains insertion order
- High performance for logging systems Example:

```
db.createCollection("logs", { capped: true, size: 100000 })
```

**Hints**:
Often used for **log storage and streaming data**.

---

## Q76. What is a TTL Index in MongoDB?

**Ans76.**
A **TTL (Time To Live) Index** automatically deletes documents after a certain period.

Example in MongoDB:

```
db.sessions.createIndex(
 { createdAt: 1 },
 { expireAfterSeconds: 3600 }
)
```

Explanation:

- Documents expire after **1 hour**.

**Hints**:
Used for:

- Sessions
- Cache data
- Temporary records

---

## Q77. What is Schema Design in MongoDB?

**Ans77.**
Schema design in MongoDB refers to organizing data in collections and documents effectively.

Approaches:

1. Embedded documents
2. Referenced documents Example:

```
{
 "name": "Rahul",
 "orders": [
  {"product": "Laptop"},
  {"product": "Keyboard"}
 ]
}
```

**Hints**:
MongoDB schema design focuses on **application query patterns**.

---

## Q78. What is Data Replication in MongoDB?

**Ans78.**
**Replication** copies data across multiple servers to ensure availability and fault tolerance.

In MongoDB, replication is implemented using **Replica Sets**.

Replica Set components:

- Primary node
- Secondary nodes

**Hints**:
Replication provides:

- High availability
- Automatic failover

---

### Q79. What is Sharding in MongoDB?

**Ans79.**
**Sharding** is the process of distributing data across multiple servers.

Used when a database becomes too large for a single server.

In MongoDB:

- Data is split across multiple **shards**
- Each shard stores part of the data

**Hints**:
Sharding enables **horizontal scaling**.

---

### Q80. What is a Replica Set in MongoDB?

**Ans80.**
A **Replica Set** is a group of MongoDB servers that maintain the same data set.

Components in MongoDB:

1. **Primary Node** – handles writes
2. **Secondary Nodes** – replicate data
3. **Arbiter** – participates in elections

**Hints**:
Replica sets ensure **high availability and automatic failover**.

# Topic 9: Security, Backup, and Storage Engines

### Q81. How do you create a user in MySQL?

**Ans81.**
A new database user can be created using the **CREATE USER** statement in MySQL.

Example:

```
CREATE USER 'appuser'@'localhost'
IDENTIFIED BY 'StrongPassword123';
```

Explanation:

- 'appuser' is the username.
- 'localhost' specifies where the user can connect from.

**Hints:**
Creating separate users improves **database security**.

---

## Q82. How do you grant privileges to a MySQL user?

**Ans82.**
Privileges are assigned using the **GRANT** statement.

Example in MySQL:

```
GRANT SELECT, INSERT, UPDATE
ON company_db.* TO
'appuser'@'localhost';
```

Explanation:

- User can read and modify data in the company_db database.

**Hints:**
Always follow the **principle of least privilege**.

---

## Q83. What is the REVOKE command?

**Ans83.**
The **REVOKE** command removes privileges from a user.

Example:

```
REVOKE INSERT
ON company_db.* FROM
'appuser'@'localhost';
```

Explanation:

- The user can no longer insert records.

**Hints:**
Useful for managing **database access control**.

---

## Q84. How do you backup a MySQL database?

**Ans84.**
Database backups can be created using the **mysqldump** utility from MySQL.

Example command:

```
mysqldump -u root -p company_db > backup.sql
```

Explanation:

- Exports the entire database to a SQL file.

**Hints:**
Backups protect data against **hardware failures and accidental deletions**.

---

## Q85. How do you restore a MySQL database?

**Ans85.**
A database can be restored using the **mysql command-line tool**.

Example:

```
mysql -u root -p company_db < backup.sql
```

Explanation:

- Restores all tables and records from the backup file.

**Hints:**
Always test backups in **staging environments**.

---

## Q86. What are Storage Engines in MySQL?

**Ans86.**
A **Storage Engine** defines how data is stored, indexed, and managed inside the database.

Common engines in MySQL:

- InnoDB
- MyISAM

- MEMORY
- CSV

**Hints:**
Different engines provide different **performance and feature sets**.

---

## Q87. What is the InnoDB Storage Engine?

**Ans87.**
**InnoDB** is the default storage engine in MySQL.

Features:

- Supports **transactions**
- Supports **foreign keys**
- Provides **row-level locking**
- Ensures **ACID compliance**

**Hints:**
InnoDB is best for **high-reliability applications**.

---

## Q88. What is MyISAM?

**Ans88.**
**MyISAM** is an older storage engine in MySQL.

Characteristics:

- Fast read performance
- No transaction support
- Table-level locking
- No foreign keys

**Hints:**
Used mainly for **read-heavy workloads**.

---

## Q89. What security features does MongoDB provide?

**Ans89.**
MongoDB provides several security features:

- Authentication
- Role-based access control

- Encryption
- Network security
- Auditing

**Hints:**
Security should always be enabled in **production environments**.

---

### Q90. What is Authentication in MongoDB?

**Ans90.**
**Authentication** verifies the identity of users accessing the database.

Example user creation in MongoDB:

```
db.createUser({  user: "admin",
pwd: "StrongPassword",  roles:
["readWrite", "dbAdmin"] })
```

Explanation:

- The user gets **read and write permissions**.

**Hints:**
Authentication prevents **unauthorized access**.

# Topic 10: Performance, Partitioning, and Monitoring

### Q91. What is Database Partitioning?

**Ans91.**
**Partitioning** divides a large database table into smaller parts called **partitions** to improve performance and manageability.

Supported in MySQL.

Example:

```
CREATE TABLE orders
(  id INT,  order_date
DATE
)
PARTITION BY RANGE (YEAR(order_date)) (
 PARTITION p0 VALUES LESS THAN (2022),
 PARTITION p1 VALUES LESS THAN (2024)
);
```

**Hints:**
Partitioning helps handle **very large tables efficiently**.

---

### Q92. What are the types of Partitioning in MySQL?

**Ans92.**

Common partitioning types in MySQL:

1. **Range Partitioning** – based on value ranges
2. **List Partitioning** – based on discrete values
3. **Hash Partitioning** – based on hash function
4. **Key Partitioning** – similar to hash but uses internal hashing

**Hints:**
Partitioning improves **query performance and maintenance**.

---

## Q93. What is Query Performance Tuning?

**Ans93.**
**Query tuning** is the process of improving database query efficiency.

Techniques:

- Using indexes
- Avoiding SELECT *
- Limiting returned rows
- Optimizing joins Used in systems like MySQL.

**Hints:**
Efficient queries reduce **database load and latency**.

---

## Q94. What is the Slow Query Log in MySQL?

**Ans94.**
The **Slow Query Log** records queries that take longer than a defined time to execute.

Available in MySQL.

Example configuration:

```
SET GLOBAL slow_query_log = 'ON';
```

**Hints:**
Helps developers **identify performance bottlenecks**.

---

## Q95. What is Write Concern in MongoDB?

**Ans95.**
**Write Concern** defines the level of acknowledgment requested from MongoDB when writing data.

Example in MongoDB:

```
db.users.insertOne(
 {name:"Rahul"},
 {writeConcern:{w:"majority"}}
)
```

Explanation:

- "majority" ensures the write is confirmed by most replica set members.

**Hints:**
Higher write concern increases **data safety**.

---

## Q96. What is Read Concern in MongoDB?

**Ans96.**
**Read Concern** controls the level of consistency for read operations.

Supported in MongoDB.

Example:

```
db.users.find().readConcern("majority")
```

Explanation:
Ensures the read returns data acknowledged by a majority of nodes.

**Hints:**
Important in **distributed systems**.

---

## Q97. What is Database Monitoring?

**Ans97.**
**Database monitoring** tracks database performance, availability, and health.

Monitoring tools for databases like:

- MySQL
- MongoDB

Metrics monitored:

- Query performance
- CPU usage
- Disk usage

- Connection count

**Hints:**
Monitoring helps prevent **performance degradation**.

---

## Q98. What is MongoDB Profiling?

**Ans98.**
**MongoDB Profiler** collects detailed information about database operations.

Enabled in MongoDB.

Example:

```
db.setProfilingLevel(1)
```

Explanation:

- Level 0 → Off
- Level 1 → Slow operations
- Level 2 → All operations

**Hints:**
Useful for **debugging slow queries**.

---

## Q99. What is Database Logging?

**Ans99.**
**Database logging** records events and operations performed in the database.

Logs include:

- Query execution
- Errors
- Connection attempts

Used in both:

- MySQL
- MongoDB

**Hints**:
Logs help in **troubleshooting and auditing**.

---

SOFTPRO INDIA

### Q100. What is Horizontal Scaling?

**Ans100.**
**Horizontal scaling** means increasing database capacity by adding more servers instead of upgrading a single server.

Common in distributed databases like MongoDB.

Example:
Using **sharding** to distribute data across multiple servers.

**Hints**:
Horizontal scaling improves:

- Scalability
- Fault tolerance
- Performance for large systems.

# Topic 10: Performance, Partitioning, and Monitoring

### Q91. What is Database Partitioning?

**Ans91.**
**Partitioning** divides a large database table into smaller parts called **partitions** to improve performance and manageability.

Supported in MySQL.

Example:

```
CREATE TABLE orders
(  id INT,
order_date DATE
)
PARTITION BY RANGE (YEAR(order_date)) (
 PARTITION p0 VALUES LESS THAN (2022),
 PARTITION p1 VALUES LESS THAN (2024)
);
```

**Hints:**
Partitioning helps handle **very large tables efficiently**.

---

### Q92. What are the types of Partitioning in MySQL?

**Ans92.**

Common partitioning types in MySQL:

1. **Range Partitioning** – based on value ranges
2. **List Partitioning** – based on discrete values
3. **Hash Partitioning** – based on hash function
4. **Key Partitioning** – similar to hash but uses internal hashing

**Hints:**
Partitioning improves **query performance and maintenance**.

---

## Q93. What is Query Performance Tuning?

**Ans93.**
**Query tuning** is the process of improving database query efficiency.

Techniques:

- Using indexes
- Avoiding `SELECT *`
- Limiting returned rows
- Optimizing joins Used in systems like MySQL.

**Hints:**
Efficient queries reduce **database load and latency**.

---

## Q94. What is the Slow Query Log in MySQL?

**Ans94.**
The **Slow Query Log** records queries that take longer than a defined time to execute.

Available in MySQL.

Example configuration:

```
SET GLOBAL slow_query_log = 'ON';
```

**Hints:**
Helps developers **identify performance bottlenecks**.

---

## Q95. What is Write Concern in MongoDB?

**Ans95.**
**Write Concern** defines the level of acknowledgment requested from MongoDB when writing data.

Example in MongoDB:

```
db.users.insertOne(
 {name:"Rahul"},
 {writeConcern:{w:"majority"}}
)
```

Explanation:

• `"majority"` ensures the write is confirmed by most replica set members.

**Hints:**
Higher write concern increases **data safety**.

---

## Q96. What is Read Concern in MongoDB?

**Ans96.**
**Read Concern** controls the level of consistency for read operations.

Supported in MongoDB.

Example:

```
db.users.find().readConcern("majority")
```

Explanation:
Ensures the read returns data acknowledged by a majority of nodes.

**Hints:**
Important in **distributed systems**.

---

## Q97. What is Database Monitoring?

**Ans97.**
**Database monitoring** tracks database performance, availability, and health.

Monitoring tools for databases like:

- MySQL
- MongoDB

Metrics monitored:

- Query performance
- CPU usage
- Disk usage

- Connection count

**Hints:**
Monitoring helps prevent **performance degradation**.

---

## Q98. What is MongoDB Profiling?

**Ans98.**
**MongoDB Profiler** collects detailed information about database operations.

Enabled in MongoDB.

Example:

```
db.setProfilingLevel(1)
```

Explanation:

- Level 0 → Off
- Level 1 → Slow operations
- Level 2 → All operations

**Hints:**
Useful for **debugging slow queries**.

---

## Q99. What is Database Logging?

**Ans99.**
**Database logging** records events and operations performed in the database.

Logs include:

- Query execution
- Errors
- Connection attempts

Used in both:

- MySQL
- MongoDB

**Hints:**
Logs help in **troubleshooting and auditing**.

---

### Q100. What is Horizontal Scaling?

**Ans100.**
**Horizontal scaling** means increasing database capacity by adding more servers instead of upgrading a single server.

Common in distributed databases like MongoDB.

Example:
Using **sharding** to distribute data across multiple servers.

**Hints:**
Horizontal scaling improves:

- Scalability
- Fault tolerance
- Performance for large systems.

# Topic 12: Database Design and Optimization

### Q111. What is Database Schema?

**Ans111.**
A **Database Schema** is the logical structure that defines how data is organized in a database.

It includes:

- Tables
- Columns
- Relationships
- Constraints
- Indexes

Used in relational systems such as MySQL.

Example schema:

```
Users Table id |
name | email

Orders Table id |
user_id | product
```

**Hints:**
Schema acts as the **blueprint of a database**.

## Q112. What is Schema Design in relational databases?

**Ans112.**
Schema design is the process of structuring tables and relationships to ensure efficient storage and retrieval of data.

Used in systems like MySQL.

Steps:

1. Identify entities (Users, Orders)
2. Define relationships
3. Normalize data
4. Define keys and constraints

**Hints:**
Good schema design improves **performance and maintainability**.

---

## Q113. What is Schema Design in MongoDB?

**Ans113.**
Schema design in MongoDB focuses on structuring documents based on application queries.

Common patterns:

- Embedded documents
- Referenced documents
- Bucket pattern
- Attribute pattern Example:

```
{
 "name":"Rahul",
 "orders":[
  {"product":"Laptop","price":70000}
 ]
}
```

**Hints**:
MongoDB schema design prioritizes **read performance**.

---

## Q114. What is the Embedding Pattern in MongoDB?

**Ans114.**
The **Embedding Pattern** stores related data inside the same document.

Example in MongoDB:

```
{
 "user":"Rahul",
 "addresses":[
  {"city":"Delhi"},
  {"city":"Lucknow"}
 ]
}
```

Advantages:

- Faster reads
- Fewer queries

**Hints**:
Best used when related data is **frequently accessed together**.

---

## Q115. What is the Referencing Pattern in MongoDB?

**Ans115.**
The **Referencing Pattern** stores related data in separate collections and references them using IDs.

Example in MongoDB:

Users Collection:

```
{
 "_id":1,
 "name":"Rahul"
}
```

Orders Collection:

```
{
 "user_id":1,
 "product":"Laptop"
}
```

**Hints**:
Used when data relationships are **complex or large**.

---

## Q116. What is Query Optimization using Indexes?

**Ans116.**
Indexes improve query performance by allowing databases to quickly locate records.

Example in MySQL:

```
CREATE INDEX idx_email
ON users(email);
```

Benefits:

- Faster SELECT queries
- Reduced table scans

**Hints**:
Indexes should be used on **frequently searched columns**.

---

## Q117. What is Database Caching?

**Ans117.**
**Database caching** stores frequently accessed data in memory to reduce database load.

Common caching tools:

- Redis
- Memcached

Benefits:

- Faster response time
- Reduced database queries

**Hints**:
Caching is widely used in **high-traffic web applications**.

---

## Q118. What is Connection Pooling?

**Ans118.**
**Connection pooling** maintains a pool of reusable database connections instead of creating a new connection for every request.

Used with databases such as:

- MySQL
- MongoDB

Benefits:

- Improved performance

- Reduced connection overhead

**Hints**:
Common in backend frameworks like Node.js and Java.

---

## Q119. What is Data Consistency?

**Ans119.**
**Data consistency** ensures that the database remains in a valid state before and after transactions.

Supported strongly in relational systems like MySQL.

Example:
When transferring money between bank accounts:

- Debit from one account
- Credit to another

**Hints**:
Maintained using **ACID properties**.

---

## Q120. What is Eventual Consistency?

**Ans120.**
**Eventual consistency** means that data updates will eventually propagate to all nodes in a distributed system.

Used in distributed databases such as MongoDB.

Example:
In a replica set, secondary nodes may receive updates slightly later than the primary.

**Hints**:
Eventual consistency prioritizes **availability and scalability**.

# Topic 1: Node.js Basics and Architecture

---

**Q1. What is Node.js and explain its core philosophy?**
**Ans1.** Node.js is an open-source, cross-platform JavaScript runtime environment that allows developers to run JavaScript code outside the browser. It is built on Chrome's V8 JavaScript engine and is designed to build scalable network applications.

Node.js follows a **non-blocking, event-driven architecture**, which makes it efficient for handling multiple requests simultaneously.

**OR**

Node.js enables developers to use JavaScript for server-side programming. Instead of creating a new thread for every request like traditional servers, Node.js uses a **single-threaded event loop** to handle many connections efficiently.

➢ **Event-Driven Architecture:** Node.js works based on events. When an event occurs (like a request or file read), a callback function is executed.
➢ **Non-Blocking I/O:** Node.js performs operations asynchronously so the server does not wait for one task to finish before starting another.
➢ **Single Threaded Model:** Node.js uses one main thread but can still handle thousands of concurrent requests.

SOFTPRO INDIA

**Hints:**
Node.js is commonly used for **real-time applications**, APIs, microservices, and streaming applications.

---

**Q2. What is the V8 Engine in Node.js?**
**Ans2.** The **V8 Engine** is the JavaScript engine developed by Google and used in Google Chrome. Node.js uses this engine to execute JavaScript code outside the browser.

The V8 engine compiles JavaScript directly into machine code, making execution very fast.

➢ **Just-In-Time (JIT) Compilation:** Converts JavaScript code into optimized machine code during runtime.
➢ **High Performance:** Makes Node.js faster compared to many other scripting runtimes.
➢ **Memory Management:** Automatically handles garbage collection.

**Hints:**
Without the V8 engine, JavaScript would only run inside web browsers.

---

**Q3. What is the Event Loop in Node.js?**
**Ans3.** The **Event Loop** is the core mechanism that allows Node.js to perform non-blocking I/O operations even though JavaScript runs on a single thread.

The event loop continuously checks the **event queue** and executes tasks when the call stack becomes empty.

**Steps of Event Loop Working:**

1. Client sends request.
2. Request enters the **event queue**.
3. Event loop pushes tasks to the **call stack**.
4. Callback functions execute once operations complete.

➢ Enables Node.js to handle thousands of concurrent connections.
➢ Prevents blocking of the main thread.

**Hints:**
Event Loop is the reason Node.js is ideal for **I/O-heavy applications**.

---

**Q4. What is npm in Node.js?**
**Ans4. npm (Node Package Manager)** is the default package manager for Node.js. It is used to install, manage, and share reusable JavaScript libraries called packages.

➢ **Install packages:**
```
npm install express
```

➢ **Global installation:**
```
npm install -g nodemon
```

➢ **Project dependency management:** Packages are stored in **package.json**.

**Hints:**
npm is the **largest software registry in the world** with millions of open-source packages.

---

**Q5. What is package.json in Node.js?**
**Ans5.** The **package.json** file is the configuration file of a Node.js project. It contains metadata about the project and lists the dependencies required.

Example structure:

```
{
  "name": "my-app",
  "version": "1.0.0",
```

```
  "description": "Node.js project",
  "main": "index.js",
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

➢ Stores project information.
➢ Manages dependencies and versions.
➢ Defines scripts like start, test, build.

**Hints:**
Created using command:

```
npm init
```

---

**Q6. What are Modules in Node.js?**
**Ans6.** Modules are reusable blocks of code in Node.js that help organize application functionality.

There are three types of modules:

➢ **Core Modules:** Built-in modules such as `fs`, `http`, `path`, `os`.
➢ **Local Modules:** Custom modules created by developers.
➢ **Third-Party Modules:** Installed using npm like `express`, `mongoose`.

Example:

```
const fs = require('fs');
```

**Hints:**
Modules help achieve **code reusability and maintainability**.

---

**Q7. What is require() in Node.js?**
**Ans7.** The **require()** function is used to import modules in Node.js.

Example:

```
const http = require('http');
```

This loads the **http module** so that we can create servers.

➢ Loads built-in modules.
➢ Loads external packages installed via npm.
➢ Loads custom modules.

**Hints:**
Node.js uses the **CommonJS module system** where require() is the main method for importing modules.

---

**Q8. What is the difference between synchronous and asynchronous programming in Node.js?**

**Ans8.**

| Feature | Synchronous | Asynchronous |
|---|---|---|
| Execution | Runs step by step | Runs tasks in background |
| Blocking | Blocking | Non-blocking |
| Performance | Slower for I/O tasks | Faster for concurrent tasks |

Example:

**Synchronous**

```
fs.readFileSync("file.txt");
```

**Asynchronous**

```
fs.readFile("file.txt", callback);
```

➢ Node.js mainly uses **asynchronous programming**.

**Hints:**
Async operations improve server performance.

---

**Q9. What is the fs module in Node.js?**

**Ans9.** The **fs (File System)** module allows Node.js to interact with the file system.

It can:

➢ Create files
➢ Read files
➢ Write files
➢ Delete files

Example:

```
const fs = require('fs');

fs.readFile('data.txt','utf8',(err,data)=>{
```

```
   console.log(data);
});
```

**Hints:**
fs module supports both **sync and async methods**.

---

**Q10. What is the http module in Node.js?**

**Ans10.** The **http module** is a built-in module that allows Node.js to create web servers.

Example:

```
const http = require('http');

const server = http.createServer((req,res)=>{
   res.write("Hello World");
   res.end();
});

server.listen(3000);
```

➢ Handles HTTP requests and responses.
➢ Used to build web servers and APIs.

**Hints:**
Frameworks like **Express.js** are built on top of the http module to simplify server development.

**Topic 2: Node.js Advanced Concepts**

---

**Q11. What are Callbacks in Node.js?**
**Ans11.** A **callback** is a function that is passed as an argument to another function and executed after that function completes its task.

Node.js uses callbacks extensively because most operations are **asynchronous**.

Example:

```
function greet(name, callback){
   console.log("Hello " + name);
   callback();
}

greet("Pushkar", function(){
   console.log("Callback executed");
});
```

➢ Helps handle asynchronous operations.
➢ Used in file operations, database queries, and API calls.

**Hints:**
Callbacks can lead to **Callback Hell** if many nested callbacks are used.

---

**Q12. What is Callback Hell?**

**Ans12. Callback Hell** occurs when multiple callbacks are nested inside each other, making the code difficult to read and maintain.

Example:

```
fs.readFile("file1.txt", function(err,data){
   fs.readFile("file2.txt", function(err,data){
      fs.readFile("file3.txt", function(err,data){
         console.log(data);
      });
   });
});
```

Problems:

➢ Difficult to read
➢ Hard to debug
➢ Hard to maintain

**Hints:**
Callback Hell can be solved using **Promises or Async/Await**.

---

**Q13. What are Promises in Node.js?**

**Ans13.** A **Promise** is an object that represents the eventual completion or failure of an asynchronous operation.

A Promise has three states:

➢ **Pending** – operation is still running
➢ **Fulfilled** – operation completed successfully
➢ **Rejected** – operation failed

Example:

```
let promise = new Promise((resolve, reject) => {
   let success = true;

   if(success){
```

```
      resolve("Task completed");
   }else{
      reject("Task failed");
   }
});

promise.then(result => console.log(result))
       .catch(error => console.log(error));
```

**Hints:**
Promises make asynchronous code more readable than callbacks.

---

**Q14. What is Async/Await in Node.js?**

**Ans14. Async/Await** is a modern way to handle asynchronous operations in Node.js. It allows developers to write asynchronous code that looks like synchronous code.

Example:

```
async function fetchData(){
   let result = await someAsyncFunction();
   console.log(result);
}
```

➢ `async` makes a function return a Promise.
➢ `await` pauses execution until the Promise resolves.

**Hints:**
Async/Await improves readability and avoids callback hell.

---

**Q15. What are Buffers in Node.js?**

**Ans15.** A **Buffer** is a temporary storage area in memory used to handle **binary data**.

Node.js uses buffers when dealing with:

➢ File streams
➢ Network streams
➢ Binary data

Example:

```
const buffer = Buffer.from("Hello");
console.log(buffer);
```

**Hints:**
Buffers help Node.js handle data streams efficiently without waiting for the entire data to load.

**Q16. What are Streams in Node.js?**

**Ans16.** Streams are objects that allow reading or writing data **piece by piece instead of loading the entire data into memory**.

Types of streams:

➢ **Readable Streams** – read data (example: file read)
➢ **Writable Streams** – write data (example: file write)
➢ **Duplex Streams** – read and write both
➢ **Transform Streams** – modify data while reading/writing

Example:

```
const fs = require('fs');

const stream = fs.createReadStream('file.txt');
stream.on('data', chunk => {
   console.log(chunk.toString());
});
```

**Hints:**
Streams are useful for **large files and video streaming**.

---

**Q17. What is EventEmitter in Node.js?**

**Ans17.** The **EventEmitter** class is used to create and handle custom events in Node.js.

Example:

```
const EventEmitter = require('events');

const emitter = new EventEmitter();

emitter.on('message', () => {
   console.log("Event triggered");
});

emitter.emit('message');
```

➢ `.on()` listens for events
➢ `.emit()` triggers events

**Hints:**
Many Node.js modules like **HTTP and Streams** use EventEmitter internally.

---

**Q18. What is the Cluster Module in Node.js?**

**Ans18.** The **Cluster module** allows Node.js to create multiple processes to handle more requests by utilizing multiple CPU cores.

Example concept:

➢ Master process creates worker processes.
➢ Workers share the same server port.

Benefits:

➢ Better performance
➢ Load distribution
➢ Uses multi-core CPUs

**Hints:**
Without clustering, Node.js runs on **only one CPU core**.

---

**Q19. What are Worker Threads in Node.js?**

**Ans19. Worker Threads** allow Node.js to perform CPU-intensive tasks in parallel using multiple threads.

Example use cases:

➢ Image processing
➢ Data encryption
➢ Heavy computations

Difference from cluster:

➢ Cluster → multiple processes
➢ Worker Threads → multiple threads within the same process

**Hints:**
Worker threads prevent blocking the main event loop.

---

**Q20. What is process in Node.js?**

**Ans20.** The **process object** is a global object that provides information about the currently running Node.js process.

Example:

```
console.log(process.version);
console.log(process.platform);
```

Common properties:

➢ `process.env` – environment variables
➢ `process.argv` – command line arguments
➢ `process.exit()` – exit program

**Hints:**
Used for **environment configuration and debugging**.

**Topic 3: Express.js Basics**

---

**Q21. What is Express.js?**
**Ans21.** Express.js is a minimal and flexible web application framework for Node.js that simplifies the process of building web servers and APIs.

It provides a robust set of features for web and mobile applications such as routing, middleware, and HTTP utilities.

**OR**

Express.js is built on top of the Node.js **http module** and provides an easier way to manage server logic and request handling.

➢ Simplifies server creation
➢ Provides routing system
➢ Supports middleware architecture
➢ Ideal for building REST APIs

**Hints:**
Express.js is one of the most popular frameworks used in the **Node.js ecosystem**.

---

**Q22. How do you install Express.js?**

**Ans22.** Express.js can be installed using **npm (Node Package Manager)**.

Steps:

1. Initialize a Node.js project

```
npm init -y
```

2. Install Express

```
npm install express
```

3. Import Express in the project

```
const express = require('express');
```

➢ Express will be added to **dependencies** in package.json.

**Hints:**
`node_modules` folder will store all installed packages.

---

**Q23. How do you create a basic server using Express.js?**

**Ans23.** A basic Express server can be created using the following code:

```
const express = require('express');

const app = express();

app.get('/', (req, res) => {
   res.send("Hello World");
});

app.listen(3000, () => {
   console.log("Server running on port 3000");
});
```

Explanation:

➢ `express()` creates an application instance
➢ `app.get()` defines a route
➢ `res.send()` sends response to client
➢ `app.listen()` starts the server

**Hints:**
Express server is easier to write compared to Node's **http module**.

---

**Q24. What is Routing in Express.js?**

**Ans24. Routing** refers to defining how an application responds to a client request for a specific endpoint (URL) and HTTP method.

Example:

```
app.get('/about', (req, res) => {
   res.send("About Page");
});
```

Types of HTTP routes:

➢ GET – retrieve data
➢ POST – send data
➢ PUT – update data
➢ DELETE – delete data

**Hints:**
Routing determines **which function runs when a user visits a URL**.

---

**Q25. What are Middleware functions in Express.js?**

**Ans25.** Middleware functions are functions that execute **between the request and response cycle**.

They have access to:

➢ Request object (`req`)
➢ Response object (`res`)
➢ Next middleware function (`next`)

Example:

```
app.use((req, res, next) => {
   console.log("Middleware executed");
   next();
});
```

Types of middleware:

➢ Application-level middleware
➢ Router-level middleware
➢ Error-handling middleware
➢ Built-in middleware

**Hints:**
Middleware is used for **logging, authentication, validation, etc.**

---

**Q26. What are Request and Response objects in Express?**

**Ans26.**

**Request Object (req):**
Represents the HTTP request and contains data sent by the client.

Common properties:

- ➢ `req.params` – route parameters
- ➢ `req.query` – query parameters
- ➢ `req.body` – request body data

**Response Object (res):**
Represents the HTTP response that the server sends to the client.

Common methods:

- ➢ `res.send()` – send response
- ➢ `res.json()` – send JSON response
- ➢ `res.status()` – set status code

Example:

```
app.get('/user', (req,res)=>{
   res.json({name:"Pushkar"});
});
```

**Hints:**
`req` and `res` are fundamental objects in Express request handling.

---

**Q27. What is express.json() middleware?**

**Ans27.** `express.json()` is a built-in middleware used to parse incoming JSON requests.

Example:

```
app.use(express.json());
```

When a client sends JSON data:

```
{
   "name":"John"
}
```

Express converts it into JavaScript object:

```
req.body.name
```

**Hints:**
Required when building **REST APIs** that accept JSON data.

---

**Q28. How do you serve static files in Express.js?**

**Ans28.** Static files such as images, CSS, and JavaScript files can be served using the **express.static() middleware**.

Example:

```
app.use(express.static('public'));
```

Folder structure:

```
project
 ├── public
 │   ├── style.css
 │   ├── script.js
```

Access example:

```
http://localhost:3000/style.css
```

**Hints:**
Used to serve **frontend assets** in Express applications.

---

**Q29. What is Express Router?**

**Ans29.** Express Router allows developers to create **modular route handlers**.

Instead of writing all routes in one file, they can be separated into multiple files.

Example:

**routes/user.js**

```
const express = require('express');
const router = express.Router();

router.get('/', (req,res)=>{
   res.send("User route");
});

module.exports = router;
```

**Main file**

```
const userRoutes = require('./routes/user');
app.use('/users', userRoutes);
```

**Hints:**
Router helps keep large applications **organized and scalable**.

---

**Q30. How does error handling work in Express.js?**

**Ans30.** Error handling in Express is done using **error-handling middleware**.

Example:

```
app.use((err, req, res, next) => {
   console.error(err.stack);
   res.status(500).send("Something went wrong");
});
```

Characteristics:

➢ Accepts **four parameters** `(err, req, res, next)`
➢ Catches errors in the application
➢ Sends appropriate response to client

**Hints:**
Error middleware must be defined **after all routes**.

**Topic 4: Express.js Routing & Middleware Deep Concepts**

---

**Q31. What are Route Parameters in Express.js?**
**Ans31.** Route parameters are used to capture dynamic values from the URL.

They are defined using a colon `:` in the route path.

Example:

```
app.get('/user/:id', (req, res) => {
   res.send("User ID: " + req.params.id);
});
```

If the URL is:

```
/user/101
```

Then:

```
req.params.id = 101
```

➢ Used to identify specific resources.
➢ Commonly used in REST APIs.

**Hints:**
Example: `/product/:productId`

---

**Q32. What are Query Parameters in Express.js?**

**Ans32.** Query parameters are key-value pairs sent in the URL after a `?`.

Example URL:

```
/search?name=pushkar&age=25
```

Accessing in Express:

```
app.get('/search', (req, res) => {
   res.send(req.query.name);
});
```

Result:

```
pushkar
```

➢ Used for filtering and searching data.
➢ Not part of the route path.

**Hints:**
`req.query` stores all query parameters.

---

**Q33. What is the difference between req.params and req.query?**

**Ans33.**

| Feature | req.params | req.query |
|---|---|---|
| Source | URL path | Query string |
| Example | `/user/:id` | `/user?id=10` |
| Usage | Identify resource | Filter/search |

Example:

```
/user/10 → req.params.id
/user?id=10 → req.query.id
```

**Hints:**
`params` → part of route
`query` → optional data in URL.

---

**Q34. What is Body Parsing in Express.js?**

**Ans34.** Body parsing refers to extracting data from the request body sent by the client.

Example request body:

```
{
   "name": "Pushkar",
   "age": 22
}
```

Express middleware:

```
app.use(express.json());
```

Then data can be accessed using:

```
req.body.name
```

➢ Required for **POST and PUT requests**.

**Hints:**
Without body parser, `req.body` will be undefined.

---

**Q35. What is Third-Party Middleware in Express?**

**Ans35.** Third-party middleware are external packages installed using npm that add additional functionality to Express applications.

Examples:

➢ **morgan** – request logging
➢ **cors** – cross-origin resource sharing
➢ **helmet** – security headers
➢ **body-parser** – request body parsing

Installation example:

```
npm install morgan
```

Usage:

```
const morgan = require('morgan');
app.use(morgan('dev'));
```

**Hints:**
Third-party middleware simplifies common tasks.

---

**Q36. What is Morgan in Express.js?**

**Ans36.** Morgan is a **HTTP request logger middleware** for Node.js.

It logs incoming requests in the terminal.

Installation:

```
npm install morgan
```

Usage:

```
const morgan = require('morgan');
app.use(morgan('dev'));
```

Example log:

```
GET /users 200 12ms
```

➢ Useful for debugging and monitoring requests.

**Hints:**
Often used during development.

---

**Q37. What is CORS in Express.js?**

**Ans37.** CORS stands for **Cross-Origin Resource Sharing**.

It allows a server to accept requests from a different domain.

Example:

Frontend:

```
http://localhost:3000
```

Backend:

```
http://localhost:5000
```

Enable CORS:

```
const cors = require('cors');
app.use(cors());
```

Installation:

```
npm install cors
```

➢ Prevents browser security issues.

**Hints:**
Required when frontend and backend run on different domains.

---

**Q38. What is Authentication Middleware in Express?**

**Ans38.** Authentication middleware verifies whether a user is authorized to access a specific route.

Example:

```
function authMiddleware(req,res,next){
   if(req.headers.token){
      next();
   }else{
      res.status(401).send("Unauthorized");
   }
}
```

Using middleware:

```
app.get('/dashboard', authMiddleware, (req,res)=>{
   res.send("Welcome to dashboard");
});
```

➢ Protects private routes.

**Hints:**
Often used with **JWT tokens**.

---

**Q39. What is Rate Limiting in Express.js?**

**Ans39.** Rate limiting restricts the number of requests a user can make to a server within a specific time period.

Used to prevent:

➢ API abuse
➢ DDoS attacks
➢ Server overload

Example using middleware:

```
npm install express-rate-limit
const rateLimit = require("express-rate-limit");

const limiter = rateLimit({
   windowMs: 15 * 60 * 1000,
   max: 100
});

app.use(limiter);
```

**Hints:**
Limits users to **100 requests per 15 minutes**.

---

**Q40. What is Helmet Middleware in Express.js?**

**Ans40.** Helmet is a middleware that helps secure Express applications by setting various HTTP headers.

Installation:

```
npm install helmet
```

Usage:

```
const helmet = require('helmet');
app.use(helmet());
```

Security benefits:

➢ Prevents **XSS attacks**
➢ Protects against **clickjacking**
➢ Secures HTTP headers

**Hints:**
Helmet improves **web application security** with minimal configuration.

**Topic 5: REST API Development with Express**

---

**Q41. What is a REST API?**
**Ans41.** A **REST API (Representational State Transfer API)** is a web service that allows communication between client and server using HTTP methods.

REST APIs follow a **stateless architecture**, meaning each request from the client contains all necessary information.

Common characteristics:

➢ Uses HTTP methods (GET, POST, PUT, DELETE)
➢ Uses URLs to identify resources
➢ Typically returns data in JSON format

Example API endpoint:

```
GET /users
```

**Hints:**
REST APIs are widely used in **web applications, mobile apps, and microservices**.

---

SOFTPRO INDIA

**Q42. What are CRUD operations in REST APIs?**

**Ans42.** CRUD represents the four basic operations performed on data.

| Operation | HTTP Method | Description |
|---|---|---|
| Create | POST | Add new data |
| Read | GET | Retrieve data |
| Update | PUT/PATCH | Modify existing data |
| Delete | DELETE | Remove data |

Example:

```
POST /users
GET /users
PUT /users/1
DELETE /users/1
```

**Hints:**
CRUD operations form the foundation of most REST APIs.

---

**Q43. How do you handle POST requests in Express.js?**

**Ans43.** POST requests are used to **send data from the client to the server**.

Example:

```
app.post('/users', (req, res) => {
   const user = req.body;
   res.send("User created");
});
```

Client request example:

```
POST /users
Content-Type: application/json

{
  "name": "Pushkar"
}
```

Requirements:

➢ Use `express.json()` middleware to parse JSON body.

**Hints:**
POST requests are used to **create new resources**.

---

**Q44. What is the difference between PUT and PATCH?**

**Ans44.**

| Feature | PUT | PATCH |
|---|---|---|
| Update Type | Full update | Partial update |
| Data Required | Entire object | Only changed fields |
| Usage | Replace resource | Modify specific field |

Example:

PUT request:

```
PUT /users/1
{
 "name":"John",
 "age":25
}
```

PATCH request:

```
PATCH /users/1
{
 "age":26
}
```

**Hints:**
PATCH is more efficient when updating only a small part of the data.

---

**Q45. How do you handle DELETE requests in Express.js?**

**Ans45.** DELETE requests remove a specific resource from the server.

Example:

```
app.delete('/users/:id', (req, res) => {
   const id = req.params.id;
   res.send("User deleted with id " + id);
});
```

Example request:

```
DELETE /users/5
```

➢ Used to remove data permanently.

**Hints:**
Always validate resource existence before deletion.

---

**Q46. What are HTTP Status Codes?**

**Ans46.** HTTP status codes indicate the result of an HTTP request.

Common status codes:

| Code | Meaning |
|---|---|
| 200 | Success |
| 201 | Resource Created |
| 400 | Bad Request |
| 401 | Unauthorized |
| 404 | Not Found |
| 500 | Server Error |

Example in Express:

```
res.status(200).json({message:"Success"});
```

**Hints:**
Using proper status codes improves **API clarity and debugging**.

---

**Q47. What is API Versioning?**

**Ans47.** API versioning is the practice of managing changes in an API without breaking existing clients.

Example:

```
/api/v1/users
/api/v2/users
```

Implementation example:

```
app.use('/api/v1', userRoutes);
```

Benefits:

➢ Backward compatibility
➢ Easier updates
➢ Better API maintenance

**Hints:**
Large companies often maintain multiple API versions.

**Q48. What is JSON in APIs?**

**Ans48.** JSON (**JavaScript Object Notation**) is a lightweight data format used for exchanging data between client and server.

Example JSON response:

```
{
  "id": 1,
  "name": "Pushkar",
  "email": "pushkar@example.com"
}
```

Sending JSON in Express:

```
res.json({message:"User created"});
```

Advantages:

➢ Human readable
➢ Lightweight
➢ Language independent

**Hints:**
Most REST APIs use **JSON as the default data format**.

---

**Q49. What are REST API Best Practices?**

**Ans49.** Best practices help design clean and scalable APIs.

Important practices:

➢ Use proper HTTP methods
➢ Use meaningful URL structure
➢ Return proper status codes
➢ Use pagination for large data
➢ Implement authentication
➢ Use versioning

Example good API structure:

```
GET /users
GET /users/10
POST /users
DELETE /users/10
```

SOFTPRO INDIA

**Hints:**
Good API design improves **maintainability and usability**.

---

**Q50. What is Postman and why is it used?**

**Ans50.** Postman is an API testing tool used to send HTTP requests to a server and view responses.

Features:

➢ Test REST APIs
➢ Send GET, POST, PUT, DELETE requests
➢ Add headers and authentication
➢ View JSON responses
➢ Save API collections

Example workflow:

1. Select HTTP method (GET/POST)
2. Enter API URL
3. Send request
4. View response

**Hints:**
Postman is widely used by **backend developers for API testing**.

**Topic 6: Node.js File System, Path, OS & Core Modules**

---

**Q51. What is the File System (fs) module in Node.js?**
**Ans51.** The **fs module** is a built-in module in Node.js used to interact with the file system.

It allows developers to create, read, update, and delete files.

Common operations:

➢ Read files
➢ Write files
➢ Create files
➢ Delete files
➢ Rename files

Example:

```
const fs = require('fs');

fs.readFile('data.txt','utf8',(err,data)=>{
  if(err) throw err;
  console.log(data);
});
```

**Hints:**

The fs module provides both **synchronous and asynchronous methods**.

---

**Q52. What is the difference between fs.readFile() and fs.readFileSync()?**

**Ans52.**

| Feature | fs.readFile() | fs.readFileSync() |
|---|---|---|
| Type | Asynchronous | Synchronous |
| Blocking | Non-blocking | Blocking |
| Performance | Better for servers | Slower for large tasks |

Example:

**Asynchronous**

```
fs.readFile('file.txt','utf8',(err,data)=>{
  console.log(data);
});
```

**Synchronous**

```
const data = fs.readFileSync('file.txt','utf8');
console.log(data);
```

**Hints:**

Node.js applications usually prefer **asynchronous methods**.

---

**Q53. How do you write a file in Node.js?**

**Ans53.** Files can be written using the **fs.writeFile()** method.

Example:

```
const fs = require('fs');
```

```
fs.writeFile('test.txt',"Hello Node.js",(err)=>{
  if(err) throw err;
  console.log("File saved");
});
```

This method:

➢ Creates a file if it does not exist
➢ Overwrites existing file content

**Hints:**
Use fs.appendFile() if you want to add content instead of replacing it.

---

**Q54. What is the Path module in Node.js?**

**Ans54.** The **path module** provides utilities for working with file and directory paths.

Example:

```
const path = require('path');

console.log(path.basename('/user/file.txt'));
```

Output:

file.txt

Common methods:

➢ path.basename() – returns file name
➢ path.dirname() – returns directory path
➢ path.extname() – returns file extension

**Hints:**
Path module ensures **cross-platform compatibility**.

---

**Q55. What is path.join() in Node.js?**

**Ans55.** path.join() combines multiple path segments into one normalized path.

Example:

```
const path = require('path');

const fullPath = path.join(__dirname,'folder','file.txt');
```

```
console.log(fullPath);
```

Advantages:

➢ Prevents path errors
➢ Works across operating systems

**Hints:**
Useful when building file paths dynamically.

---

**Q56. What is the OS module in Node.js?**

**Ans56.** The **os module** provides information about the operating system.

Example:

```
const os = require('os');

console.log(os.platform());
console.log(os.cpus());
```

Common methods:

➢ os.platform() – OS platform
➢ os.cpus() – CPU information
➢ os.totalmem() – total memory
➢ os.freemem() – free memory

**Hints:**
Used for **system monitoring and diagnostics**.

---

**Q57. What are Environment Variables in Node.js?**

**Ans57.** Environment variables store configuration values outside the application code.

They are accessed using:

```
process.env
```

Example:

```
console.log(process.env.PORT);
```

Often stored in .env file:

```
PORT=3000
DB_PASSWORD=123456
```

Using dotenv package:

```
require('dotenv').config();
```

**Hints:**
Environment variables keep **sensitive data secure**.

---

**Q58. What is the Child Process module in Node.js?**

**Ans58.** The **child_process module** allows Node.js to create new processes and execute system commands.

Example:

```
const { exec } = require('child_process');

exec('dir', (err,stdout,stderr)=>{
  console.log(stdout);
});
```

Uses:

➢ Running shell commands
➢ Running external scripts
➢ CPU intensive operations

**Hints:**
Helps Node.js perform tasks outside the main process.

---

**Q59. What is spawn() in child_process?**

**Ans59.** spawn() is used to create a new process and stream data between processes.

Example:

```
const { spawn } = require('child_process');

const process = spawn('node',['app.js']);

process.stdout.on('data',(data)=>{
  console.log(data.toString());
});
```

Advantages:

➢ Efficient for large data
➢ Uses streams

**Hints:**
Preferred when handling **continuous data streams**.

---

**Q60. What is exec() in child_process?**

**Ans60.** exec() executes a command in a shell and buffers the output.

Example:

```
const { exec } = require('child_process');

exec('node -v',(err,stdout,stderr)=>{
  console.log(stdout);
});
```

Difference between spawn() and exec():

| Feature | spawn() | exec() |
|---|---|---|
| Output | Streams | Buffer |
| Large Data | Better | Not ideal |
| Use case | Continuous data | Simple commands |

**Hints:**
Use exec() for **small outputs** and spawn() for **large data streams**.

**Topic 7: Node.js Streams, Buffers & Events Deep Concepts**

---

**Q61. What are Streams in Node.js?**
**Ans61.** Streams are objects that allow data to be processed **chunk by chunk instead of loading the entire data into memory**.

This makes Node.js efficient for handling large files or continuous data.

Example:

```
const fs = require('fs');

const stream = fs.createReadStream('largeFile.txt');
```

```
stream.on('data', chunk => {
   console.log(chunk.toString());
});
```

Advantages:

➢ Memory efficient
➢ Faster processing for large files
➢ Suitable for real-time applications

**Hints:**
Streams are widely used in **file processing, video streaming, and networking**.

---

**Q62. What are the different types of Streams in Node.js?**

**Ans62.** There are four main types of streams:

| Stream Type | Description |
|---|---|
| Readable Stream | Used to read data |
| Writable Stream | Used to write data |
| Duplex Stream | Can read and write data |
| Transform Stream | Modify data while reading/writing |

Examples:

➢ `fs.createReadStream()` – Readable
➢ `fs.createWriteStream()` – Writable

**Hints:**
Streams allow Node.js to handle **large data efficiently**.

---

**Q63. What is a Readable Stream?**

**Ans63.** A **Readable Stream** is used to read data from a source.

Example:

```
const fs = require('fs');

const readableStream = fs.createReadStream('file.txt');

readableStream.on('data', chunk => {
   console.log(chunk.toString());
});
```

Events in readable streams:

- ➢ `data` – when data chunk is available
- ➢ `end` – when stream ends
- ➢ `error` – if error occurs

**Hints:**
Readable streams are used when **reading large files**.

---

**Q64. What is a Writable Stream?**

**Ans64.** A **Writable Stream** is used to write data to a destination.

Example:

```
const fs = require('fs');

const writableStream = fs.createWriteStream('output.txt');

writableStream.write("Hello Node.js");
writableStream.end();
```

Events in writable streams:

- ➢ `finish` – writing completed
- ➢ `error` – if error occurs

**Hints:**
Writable streams are useful for **saving large data efficiently**.

---

**Q65. What is pipe() in Node.js Streams?**

**Ans65.** The **pipe() method** connects a readable stream directly to a writable stream.

Example:

```
const fs = require('fs');

const readStream = fs.createReadStream('input.txt');
const writeStream = fs.createWriteStream('output.txt');

readStream.pipe(writeStream);
```

Advantages:

➢ Simplifies stream handling
➢ Automatically manages data flow
➢ Reduces memory usage

**Hints:**
Commonly used for **file copying and streaming data**.

---

**Q66. What is Backpressure in Streams?**

**Ans66. Backpressure** occurs when a writable stream cannot process incoming data as fast as it is received.

This can cause memory overflow if not handled properly.

Node.js handles backpressure automatically using:

➢ `stream.pause()`
➢ `stream.resume()`
➢ internal buffering

Example concept:

If data producer is faster than data consumer, backpressure slows the producer.

**Hints:**
Backpressure helps maintain **efficient memory management**.

---

**Q67. What are Buffers in Node.js?**

**Ans67.** A **Buffer** is a temporary memory area used to store **binary data**.

Buffers are useful when working with:

➢ File streams
➢ Network data
➢ Binary files

Example:

```
const buffer = Buffer.from("Hello");

console.log(buffer);
```

Output:

```
<Buffer 48 65 6c 6c 6f>
```

**Hints:**
Buffers allow Node.js to handle **binary data efficiently**.

---

**Q68. What is Buffer.alloc() in Node.js?**

**Ans68.** `Buffer.alloc()` creates a new buffer with a specified size.

Example:

```
const buffer = Buffer.alloc(10);

console.log(buffer);
```

This creates a buffer with **10 bytes of memory**.

Advantages:

➢ Safe memory allocation
➢ Prevents security issues

**Hints:**
`Buffer.alloc()` initializes memory with **zero values**.

---

**Q69. What is Event-Driven Architecture in Node.js?**

**Ans69.** Node.js follows an **event-driven architecture**, where actions trigger events and corresponding handlers execute.

Example:

```
const EventEmitter = require('events');

const emitter = new EventEmitter();

emitter.on('greet', () => {
   console.log("Hello Event");
});

emitter.emit('greet');
```

Process:

1. Event occurs
2. Listener detects event
3. Callback function executes

**Hints:**
Event-driven programming is the foundation of **Node.js asynchronous behavior**.

---

**Q70. What is the Event Loop in Node.js?**

**Ans70.** The **Event Loop** is the mechanism that allows Node.js to handle multiple asynchronous operations using a single thread.

Working process:

1. Requests enter the **event queue**
2. Event loop checks **call stack**
3. Executes callbacks when stack is empty

Phases of Event Loop:

➢ Timers
➢ I/O callbacks
➢ Idle/prepare
➢ Poll
➢ Check
➢ Close callbacks

**Hints:**
The event loop enables Node.js to handle **thousands of concurrent connections efficiently**.

**Topic 8: Authentication, Security & Sessions in Node.js**

---

**Q71. What is Authentication in web applications?**
**Ans71.** Authentication is the process of verifying the identity of a user before allowing access to a system or resource.

Example process:

1. User enters username and password
2. Server verifies credentials
3. If valid, user is granted access

Common authentication methods:

➢ Password-based authentication
➢ Token-based authentication
➢ OAuth authentication

**Hints:**
Authentication answers the question: **"Who are you?"**

---

**Q72. What is Authorization?**

**Ans72.** Authorization determines what actions a user is allowed to perform after authentication.

Example:

| Role | Permissions |
|---|---|
| Admin | Create, update, delete users |
| User | Read data only |

Example in Express:

```
function checkAdmin(req,res,next){
   if(req.user.role === "admin"){
      next();
   }else{
      res.status(403).send("Forbidden");
   }
}
```

**Hints:**
Authorization answers the question: **"What can you do?"**

---

**Q73. What is JWT (JSON Web Token)?**

**Ans73.** JWT is a token-based authentication method used to securely transmit information between client and server.

Structure of JWT:

```
Header.Payload.Signature
```

Example usage:

1. User logs in
2. Server generates JWT token
3. Client stores token
4. Client sends token with future requests

Example token verification:

```
const jwt = require('jsonwebtoken');

jwt.verify(token, secretKey);
```

**Hints:**
JWT is commonly used in **REST APIs and microservices**.

---

**Q74. What is bcrypt in Node.js?**

**Ans74. bcrypt** is a library used to hash passwords before storing them in the database.

This improves security by preventing plain-text password storage.

Installation:

```
npm install bcrypt
```

Example:

```
const bcrypt = require('bcrypt');

const hash = await bcrypt.hash("password",10);
```

Password verification:

```
bcrypt.compare(password, hash);
```

**Hints:**
Never store **plain-text passwords** in a database.

---

**Q75. What are Cookies in web applications?**

**Ans75.** Cookies are small pieces of data stored in the user's browser by the server.

They are used to:

➢ Maintain user sessions
➢ Store login information
➢ Track user behavior

Example in Express:

```
res.cookie("username","Pushkar");
```

Reading cookie:

```
req.cookies.username
```

**Hints:**
Cookies are automatically sent with every HTTP request to the server.

---

**Q76. What is express-session?**

**Ans76. express-session** is middleware used to manage user sessions in Express applications.

Installation:

```
npm install express-session
```

Example:

```
const session = require('express-session');

app.use(session({
   secret: 'secret-key',
   resave: false,
   saveUninitialized: true
}));
```

Advantages:

➢ Maintains login state
➢ Stores session data on server

**Hints:**
Session IDs are stored in browser cookies.

---

**Q77. What is the difference between Session Authentication and Token Authentication?**

**Ans77.**

| Feature | Session Authentication | Token Authentication |
|---|---|---|
| Storage | Server | Client |
| State | Stateful | Stateless |
| Scalability | Limited | Highly scalable |
| Example | express-session | JWT |

Explanation:

➢ Session authentication stores session data on the server.
➢ Token authentication stores authentication tokens on the client.

**Hints:**
JWT-based authentication is commonly used for **REST APIs**.

---

**Q78. What is CSRF (Cross-Site Request Forgery)?**

**Ans78.** CSRF is a security attack where a malicious website tricks a user into performing unwanted actions on another site where they are authenticated.

Example attack:

1. User logged into bank website
2. Malicious site sends hidden request
3. Bank server processes request because user session exists

Prevention methods:

➢ CSRF tokens
➢ SameSite cookies
➢ Double submit cookies

**Hints:**
CSRF protection is important for **session-based applications**.

---

**Q79. What is OAuth?**

**Ans79.** OAuth is an authentication protocol that allows users to log in using third-party services.

Example:

➢ Login with Google
➢ Login with Facebook
➢ Login with GitHub

OAuth process:

1. User clicks "Login with Google"
2. Redirect to Google authentication
3. Google verifies user
4. Token returned to application

**Hints:**
OAuth improves user experience by avoiding new account creation.

---

**Q80. What is a Secure Authentication Flow?**

**Ans80.** A secure authentication flow ensures safe login and data protection.

Typical secure process:

1. User submits credentials
2. Password hashed using bcrypt
3. Server verifies credentials
4. JWT token generated
5. Token stored in client
6. Client sends token in Authorization header

Example header:

```
Authorization: Bearer token_here
```

Security best practices:

➢ Use HTTPS
➢ Hash passwords
➢ Use JWT expiration
➢ Validate input

**Hints:**
Security is critical for protecting **user data and applications**.

**Topic 9: Database Integration with Node.js (MongoDB & SQL)**

---

**Q81. How do you connect MongoDB with Node.js?**
**Ans81.** MongoDB can be connected to a Node.js application using the **MongoDB driver** or **Mongoose ODM**.

Installation:

```
npm install mongoose
```

Connection example:

```
const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/mydb')
.then(()=> console.log("Database connected"))
.catch(err => console.log(err));
```

➢ `mongoose.connect()` establishes connection with MongoDB.
➢ Database operations can be performed after connection.

**Hints:**
MongoDB is widely used with Node.js in the **MEAN/MERN stack**.

---

**Q82. What is Mongoose in Node.js?**

**Ans82. Mongoose** is an Object Data Modeling (ODM) library for MongoDB and Node.js.

It provides a structured way to interact with MongoDB.

Features:

➢ Schema-based data modeling
➢ Data validation
➢ Middleware support
➢ Query building

Example import:

```
const mongoose = require('mongoose');
```

**Hints:**
Mongoose simplifies working with MongoDB compared to the native driver.

---

**Q83. What is a Schema in Mongoose?**

**Ans83.** A **Schema** defines the structure of documents stored in a MongoDB collection.

Example:

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
   name: String,
   email: String,
   age: Number
});
```

Benefits:

➢ Data structure definition
➢ Validation rules
➢ Default values

**Hints:**
Schemas enforce **data consistency**.

**Q84. What is a Model in Mongoose?**

**Ans84.** A **Model** is a wrapper around a schema that provides methods to interact with the database.

Example:

```
const User = mongoose.model('User', userSchema);
```

Using the model:

```
const user = new User({name:"Pushkar",age:22});
await user.save();
```

Functions provided by models:

➢ Create documents
➢ Read documents
➢ Update documents
➢ Delete documents

**Hints:**
Models represent **collections in MongoDB**.

---

**Q85. How do you create a document in MongoDB using Mongoose?**

**Ans85.** Documents can be created using `.save()` or `.create()` methods.

Example:

```
const user = new User({
   name: "Pushkar",
   age: 22
});

await user.save();
```

OR

```
await User.create({
   name: "Pushkar",
   age: 22
});
```

➢ Adds a new record to the database.

**Hints:**
Equivalent to **INSERT operation** in SQL.

---

**Q86. How do you read data from MongoDB using Mongoose?**

**Ans86.** Data can be retrieved using methods like `find()`, `findOne()`, or `findById()`.

Example:

```
const users = await User.find();
```

Find single record:

```
const user = await User.findOne({name:"Pushkar"});
```

Find by ID:

```
const user = await User.findById(id);
```

**Hints:**
These methods return **JavaScript objects** representing database documents.

---

**Q87. How do you update documents in MongoDB using Mongoose?**

**Ans87.** Documents can be updated using `updateOne()`, `updateMany()`, or `findByIdAndUpdate()`.

Example:

```
await User.updateOne(
   {name:"Pushkar"},
   {age:23}
);
```

Update by ID:

```
await User.findByIdAndUpdate(id,{age:23});
```

➢ Updates existing records.

**Hints:**
Similar to **UPDATE query in SQL**.

---

**Q88. How do you delete documents in MongoDB using Mongoose?**

**Ans88.** Documents can be deleted using `deleteOne()`, `deleteMany()`, or `findByIdAndDelete()`.

Example:

```
await User.deleteOne({name:"Pushkar"});
```

Delete by ID:

```
await User.findByIdAndDelete(id);
```

➢ Removes records from database.

**Hints:**
Equivalent to **DELETE operation in SQL**.

---

**Q89. What is the difference between SQL and NoSQL databases?**

**Ans89.**

| Feature | SQL | NoSQL |
|---|---|---|
| Structure | Tables | Collections |
| Schema | Fixed | Flexible |
| Examples | MySQL, PostgreSQL | MongoDB, Cassandra |
| Data Format | Rows and columns | JSON-like documents |

Explanation:

➢ SQL databases use structured schemas.
➢ NoSQL databases provide flexible data storage.

**Hints:**
MongoDB is a **document-oriented NoSQL database**.

---

**Q90. What is the difference between ORM and ODM?**

**Ans90.**

| Feature | ORM | ODM |
|---|---|---|
| Full form | Object Relational Mapping | Object Document Mapping |
| Database type | SQL databases | NoSQL databases |
| Example | Sequelize, TypeORM | Mongoose |

Explanation:

➢ ORM maps objects to relational tables.
➢ ODM maps objects to document databases.

**Hints:**
Mongoose is an **ODM used for MongoDB**.

**Topic 10: Performance, Scaling & Deployment in Node.js**

---

**Q91. What is Performance Optimization in Node.js?**
**Ans91.** Performance optimization refers to improving the speed, efficiency, and scalability of a Node.js application.

Common techniques:

➢ Use asynchronous operations
➢ Avoid blocking the event loop
➢ Use caching mechanisms
➢ Implement clustering
➢ Optimize database queries

Example of non-blocking code:

```
fs.readFile("file.txt",(err,data)=>{
   console.log(data);
});
```

**Hints:**
Efficient use of the **event loop** is key to Node.js performance.

---

**Q92. What is Caching in Node.js?**

**Ans92.** Caching stores frequently used data in memory so that it can be retrieved quickly without querying the database repeatedly.

Benefits:

➢ Reduces database load
➢ Improves response time
➢ Enhances scalability

Example concept:

```
let cache = {};

app.get('/data', (req,res)=>{
   if(cache.data){
      return res.send(cache.data);
   }
   const data = "database result";
   cache.data = data;
   res.send(data);
});
```

**Hints:**
Common caching tools include **Redis and Memcached**.

---

**Q93. What is Load Balancing?**

**Ans93.** Load balancing distributes incoming network traffic across multiple servers to ensure high availability and performance.

Advantages:

➢ Prevents server overload
➢ Improves reliability
➢ Enables horizontal scaling

Example architecture:

```
Client → Load Balancer → Server1
                       → Server2
                       → Server3
```

**Hints:**
Tools like **Nginx** are commonly used as load balancers.

---

**Q94. What is Clustering in Node.js?**

**Ans94.** Clustering allows Node.js applications to use multiple CPU cores by creating multiple worker processes.

Example concept:

```
Master Process
   ├── Worker 1
   ├── Worker 2
   ├── Worker 3
```

Benefits:

➢ Improves performance
➢ Handles more concurrent users
➢ Uses multi-core processors

Example module:

```
const cluster = require('cluster');
```

**Hints:**
Without clustering, Node.js runs on **a single CPU core**.

---

**Q95. What is PM2 in Node.js?**

**Ans95.** PM2 is a **process manager** used to manage and monitor Node.js applications in production.

Installation:

```
npm install pm2 -g
```

Start application:

```
pm2 start app.js
```

Features:

➢ Automatic restart on crash
➢ Load balancing
➢ Monitoring tools
➢ Log management

**Hints:**
PM2 is widely used for **production deployment**.

---

**Q96. What is Logging in Node.js applications?**

**Ans96.** Logging records application events and errors for debugging and monitoring.

Example simple logging:

```
console.log("Server started");
```

Advanced logging tools:

➢ **Winston**
➢ **Morgan**
➢ **Pino**

Example using Morgan:

```
app.use(morgan('dev'));
```

**Hints:**
Proper logging helps identify **bugs and performance issues**.

---

**Q97. What is Docker and how is it used with Node.js?**

**Ans97.** Docker is a containerization platform used to package applications and their dependencies into containers.

Benefits:

➢ Consistent environments
➢ Easy deployment
➢ Scalable infrastructure

Example Dockerfile:

```
FROM node:18
WORKDIR /app
COPY . .
RUN npm install
CMD ["node","app.js"]
```

**Hints:**
Docker ensures the application runs the same way in **development and production**.

---

**Q98. What is Environment-Based Configuration?**

**Ans98.** Environment-based configuration allows applications to run differently in development, testing, and production environments.

Example:

```
development
production
testing
```

Using environment variables:

```
if(process.env.NODE_ENV === "production"){
   console.log("Running in production");
}
```

**Hints:**
Environment configuration improves **security and flexibility**.

---

**Q99. What are Best Practices for Deploying Node.js Applications?**

**Ans99.** Important deployment best practices include:

➢ Use environment variables
➢ Implement proper error handling
➢ Use process managers like PM2
➢ Enable logging and monitoring
➢ Secure APIs with authentication
➢ Use HTTPS

Example start script in package.json:

```
"scripts": {
  "start": "node app.js"
}
```

**Hints:**
Production deployments should focus on **security, scalability, and monitoring**.

---

**Q100. What is Horizontal vs Vertical Scaling?**

**Ans100.**

| Type | Description |
|---|---|
| Vertical Scaling | Increasing server resources (CPU, RAM) |
| Horizontal Scaling | Adding more servers |

Example:

Vertical:

```
1 server → more RAM/CPU
```

Horizontal:

```
1 server → multiple servers
```

Explanation:

➢ Vertical scaling upgrades hardware.
➢ Horizontal scaling distributes load across servers.

**Hints:**
Modern cloud applications prefer **horizontal scaling**.

**Q101. How does memory management work in Node.js?**
**Ans101.** Memory management in Node.js is handled automatically by the JavaScript engine through **garbage collection**.

When variables or objects are no longer referenced in the program, the system frees that memory.

Example:

```
let user = {name:"Pushkar"};
user = null;
```

After setting `user` to `null`, the object becomes eligible for garbage collection.

Key points:

➢ Automatic memory allocation
➢ Automatic memory cleanup
➢ Prevents memory leaks if used properly

**Hints:**
Memory leaks can occur when unused objects are still referenced.

---

**Q102. What is Garbage Collection in Node.js?**

**Ans102.** Garbage collection is the process of automatically freeing memory that is no longer being used by the application.

Node.js uses a **mark-and-sweep algorithm**.

Process:

1. Identify active objects in memory
2. Mark objects that are still referenced
3. Remove unreferenced objects

Advantages:

➢ Prevents memory leaks
➢ Improves application stability

**Hints:**
Developers do not manually free memory in JavaScript.

---

**Q103. What is the Node.js Architecture?**

**Ans103.** Node.js follows a **single-threaded, event-driven, non-blocking architecture**.

Architecture components:

➢ Event Loop
➢ Callback Queue
➢ Thread Pool
➢ Event Queue

Process flow:

```
Client Request → Event Queue → Event Loop → Callback Execution
```

Advantages:

➢ Handles thousands of concurrent requests
➢ Efficient resource usage

**Hints:**
Node.js is designed for **high-performance network applications**.

---

**Q104. What is the Thread Pool in Node.js?**

**Ans104.** The **thread pool** is a group of worker threads used by Node.js to perform certain tasks that cannot be handled by the event loop alone.

Examples of tasks handled by thread pool:

➢ File system operations
➢ DNS lookups
➢ Compression
➢ Cryptography

Default size:

```
4 threads
```

It can be increased using:

```
UV_THREADPOOL_SIZE=8
```

**Hints:**
Thread pool is managed by the **libuv library**.

---

**Q105. What is Libuv in Node.js?**

**Ans105. Libuv** is a multi-platform C library used by Node.js to handle asynchronous I/O operations.

Functions of libuv:

➢ Event loop implementation
➢ Thread pool management
➢ File system operations
➢ Networking

Architecture:

```
Node.js
   ↓
Libuv
   ↓
Operating System
```

**Hints:**
Libuv is responsible for **non-blocking I/O behavior** in Node.js.

---

**Q106. What are Microtasks and Macrotasks in Node.js?**

**Ans106.**

| Type | Description | Examples |
|---|---|---|
| Microtasks | Executed immediately after current task | Promise.then(), process.nextTick() |
| Macrotasks | Executed in next event loop cycle | setTimeout(), setInterval() |

Example:

```
setTimeout(()=>console.log("Macrotask"),0);

Promise.resolve().then(()=>{
   console.log("Microtask");
});
```

Output:

```
Microtask
Macrotask
```

**Hints:**
Microtasks have **higher priority than macrotasks**.

---

**Q107. What is process.nextTick() in Node.js?**

**Ans107.** `process.nextTick()` schedules a callback to execute immediately after the current operation but before the event loop continues.

Example:

```
process.nextTick(()=>{
   console.log("Next Tick");
});
```

Characteristics:

➢ Executes before I/O events
➢ Higher priority than timers

**Hints:**
Overusing `process.nextTick()` can block the event loop.

---

**Q108. What is the difference between setImmediate() and setTimeout()?**

**Ans108.**

| Feature | setImmediate() | setTimeout() |
|---|---|---|
| Execution | After current I/O cycle | After specified delay |
| Timing | Immediate execution in next cycle | Delay-based execution |

Example:

```
setImmediate(()=>{
   console.log("Immediate");
});

setTimeout(()=>{
   console.log("Timeout");
},0);
```

**Hints:**
`setImmediate()` is commonly used for **post-I/O operations**.

---

**Q109. What is the difference between Process and Thread?**

**Ans109.**

| Feature | Process | Thread |
|---|---|---|
| Definition | Independent program | Lightweight execution unit |
| Memory | Separate memory space | Shared memory |
| Communication | IPC required | Easier communication |

Explanation:

➢ Node.js runs a **single process** with multiple asynchronous operations.
➢ Worker threads can be used for parallel tasks.

**Hints:**
Processes are heavier than threads.

---

**Q110. Why is Node.js considered non-blocking?**

**Ans110.** Node.js is considered **non-blocking** because it performs I/O operations asynchronously without stopping the execution of other tasks.

Example:

```
fs.readFile("file.txt",(err,data)=>{
   console.log(data);
});

console.log("Next task running");
```

Output:

```
Next task running
(file data appears later)
```

Explanation:

➢ The server continues processing other requests while waiting for I/O operations to complete.

**Hints:**
Non-blocking architecture enables Node.js to **handle many simultaneous connections efficiently**.

**Q111. What is the execution order of middleware in Express.js?**
**Ans111.** Middleware in Express.js executes in the **order in which it is defined** in the application.

Example:

```
app.use((req,res,next)=>{
   console.log("Middleware 1");
   next();
});

app.use((req,res,next)=>{
   console.log("Middleware 2");
   next();
});

app.get('/',(req,res)=>{
   res.send("Home Page");
});
```

Output order:

```
Middleware 1
Middleware 2
Home Page
```

Key points:

- ➢ Middleware runs sequentially
- ➢ `next()` passes control to the next middleware
- ➢ Without `next()`, the request stops

**Hints:**
Middleware order is critical in Express applications.

---

**Q112. What is API Pagination?**

**Ans112.** Pagination is the technique of dividing large datasets into smaller pages to improve performance and usability.

Example API:

```
GET /users?page=1&limit=10
```

Implementation example:

```
app.get('/users',(req,res)=>{
   const page = parseInt(req.query.page) || 1;
   const limit = parseInt(req.query.limit) || 10;

   const start = (page - 1) * limit;
   const end = page * limit;

   res.send(users.slice(start,end));
});
```

Benefits:

➢ Faster responses
➢ Reduced server load
➢ Better user experience

**Hints:**
Common pagination methods include **offset-based and cursor-based pagination**.

---

**Q113. How do you upload files in Express.js?**

**Ans113.** File uploads can be handled using the **multer middleware**.

Installation:

```
npm install multer
```

Example:

```
const multer = require('multer');

const upload = multer({dest:'uploads/'});

app.post('/upload', upload.single('file'), (req,res)=>{
   res.send("File uploaded");
});
```

Features:

➢ Supports file storage
➢ Handles multipart/form-data
➢ Allows file size limits

**Hints:**
Multer is widely used for **handling file uploads in Express**.

---

**Q114. What is centralized error handling in Express?**

**Ans114.** Centralized error handling means handling all application errors in a single middleware.

Example:

```
app.use((err,req,res,next)=>{
   console.error(err.message);
   res.status(500).json({error:"Server error"});
});
```

Advantages:

➢ Cleaner code
➢ Consistent error responses
➢ Easier debugging

**Hints:**
Error middleware must have **four parameters**.

---

**Q115. What is Input Validation in APIs?**

**Ans115.** Input validation ensures that data sent by the client meets specific rules before processing.

Example validation:

```
if(!req.body.email){
   return res.status(400).send("Email required");
}
```

Popular validation libraries:

➢ **express-validator**
➢ **Joi**
➢ **Yup**

Benefits:

➢ Prevents invalid data
➢ Improves security
➢ Protects database integrity

**Hints:**
Validation should occur **before database operations**.

---

**Q116. What is express-validator?**

**Ans116. express-validator** is a middleware library used for validating and sanitizing request data.

Installation:

```
npm install express-validator
```

Example:

```
const { body, validationResult } = require('express-validator');

app.post('/user',
   body('email').isEmail(),
   (req,res)=>{
      const errors = validationResult(req);
      if(!errors.isEmpty()){
         return res.status(400).json({errors:errors.array()});
      }
      res.send("Valid data");
   }
);
```

**Hints:**
express-validator integrates easily with **Express routes**.

---

**Q117. What is API Rate Limiting?**

**Ans117.** API rate limiting restricts the number of requests a client can make within a given time.

Example using middleware:

```
const rateLimit = require("express-rate-limit");

const limiter = rateLimit({
   windowMs: 15 * 60 * 1000,
   max: 100
});

app.use(limiter);
```

Benefits:

➢ Prevents abuse
➢ Protects server resources
➢ Improves system stability

**Hints:**
Rate limiting is important for **public APIs**.

---

**Q118. What is the difference between REST and GraphQL?**

**Ans118.**

| Feature | REST | GraphQL |
|---|---|---|
| Data fetching | Multiple endpoints | Single endpoint |
| Overfetching | Possible | Avoided |

| Feature | REST | GraphQL |
|---|---|---|
| Flexibility | Limited | High |

Example:

REST:

```
GET /users
GET /users/1/posts
```

GraphQL:

```
query{
   user(id:1){
      name
      posts{
         title
      }
   }
}
```

**Hints:**
GraphQL allows clients to request **only the required data**.

---

**Q119. What is API Documentation?**

**Ans119.** API documentation describes how developers can use an API.

It includes:

➢ Endpoints
➢ Request parameters
➢ Response formats
➢ Authentication methods

Popular tools:

➢ **Swagger (OpenAPI)**
➢ **Postman Documentation**
➢ **Redoc**

Example documentation entry:

```
GET /users
Returns list of users
```

**Hints:**
Good documentation improves **developer experience**.

**Q120. What is Swagger in Node.js?**

**Ans120.** Swagger is a tool used to create interactive API documentation based on the **OpenAPI specification**.

Installation example:

```
npm install swagger-ui-express
```

Example usage:

```
const swaggerUi = require('swagger-ui-express');

app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerDocument));
```

Advantages:

➢ Interactive API testing
➢ Clear API documentation
➢ Easy integration with Express

**Hints:**
Swagger allows developers to **test APIs directly from the browser**.

SOFTPRO INDIA

# React Js

## Q1. What is React JS?

**Ans1.**
React JS is a JavaScript library used for building user interfaces, especially for single-page applications (SPAs). It was developed and maintained by Meta and is widely used for creating fast and interactive web applications.

React allows developers to build reusable UI components and efficiently update the UI when data changes.

**Key Points:**

➢ **Component-Based Architecture:** UI is divided into small reusable components.
➢ **Virtual DOM:** React updates only the changed parts of the UI instead of reloading the entire page.
➢ **Declarative UI:** Developers describe what the UI should look like based on the state.

**Hints:**
React improves performance and maintainability by breaking UI into reusable components.

---

## Q2. What are the main features of React?

**Ans2.**

React provides several powerful features that make frontend development easier.

**Main Features:**

➢ **JSX (JavaScript XML):** Allows writing HTML-like code inside JavaScript.
➢ **Components:** Reusable pieces of UI logic.
➢ **Virtual DOM:** Improves performance by minimizing direct DOM manipulation.
➢ **One-way Data Binding:** Data flows from parent to child components.
➢ **Hooks:** Allow functional components to use state and lifecycle features.

**Hints:**
React is known for its fast rendering and reusable component architecture.

---

## Q3. What is JSX?

**Ans3.**

JSX stands for **JavaScript XML**. It is a syntax extension for JavaScript that allows developers to write HTML-like code inside JavaScript.

Example:

```
const element = <h1>Hello World</h1>;
```

JSX is not understood directly by browsers. It is converted into JavaScript using a compiler like **Babel**.

Example Conversion:

```
React.createElement("h1", null, "Hello World");
```

**Key Points:**

➢ Makes code easier to read and write
➢ Combines HTML and JavaScript logic
➢ Must return a single parent element

**Hints:**
JSX helps developers create UI in a more readable way.

---

## Q4. What is a Component in React?

**Ans4.**

A **Component** is a reusable piece of UI in React. Components allow developers to divide the interface into independent and reusable parts.

There are two types of components:

### 1. Functional Components

```
function Welcome() {
  return <h1>Hello User</h1>;
}
```

### 2. Class Components

```
class Welcome extends React.Component {
  render() {
    return <h1>Hello User</h1>;
  }
}
```

**Key Points:**

➢ Reusable UI blocks
➢ Can contain logic and styling
➢ Accept inputs called **props**

**Hints:**
Modern React mainly uses **functional components with hooks**.

---

## Q5. What is Virtual DOM?

**Ans5.**

The **Virtual DOM** is a lightweight copy of the real DOM.

When a component state changes:

1. React creates a new Virtual DOM.
2. It compares it with the previous Virtual DOM (Diffing).
3. Only the changed elements are updated in the real DOM.

**Benefits:**

➢ Faster rendering
➢ Improved performance
➢ Reduced direct DOM manipulation

**Hints:**
React does not update the whole page, only the changed parts.

---

## Q6. What is the difference between Real DOM and Virtual DOM?

**Ans6.**

| Feature | Real DOM | Virtual DOM |
|---|---|---|
| Definition | Actual DOM in the browser | Lightweight copy of the DOM |
| Speed | Slower updates | Faster updates |
| Updates | Updates entire DOM | Updates only changed elements |
| Performance | Lower | Higher |

**Hints:**
Virtual DOM makes React applications faster and more efficient.

---

## Q7. What are Props in React?

**Ans7.**

**Props (Properties)** are used to pass data from a parent component to a child component.

Props are **read-only** and cannot be modified by the receiving component.

Example:

```
function Welcome(props) {
  return <h1>Hello {props.name}</h1>;
}

<Welcome name="Pushkar" />
```

**Key Points:**

➢ Used for communication between components
➢ Immutable (cannot be changed)
➢ Passed from parent to child

**Hints:**
Props make components dynamic and reusable.

---

## Q8. What is State in React?

**Ans8.**

**State** is an object that stores dynamic data in a component and controls the behavior of the component.

When the state changes, React automatically re-renders the component.

Example:

```
const [count, setCount] = useState(0);
```

**Key Points:**

➢ Used to store dynamic data
➢ Changing state re-renders the component
➢ Managed inside the component

**Hints:**
Props are external data, while **state is internal data**.

---

## Q9. What is the difference between Props and State?

**Ans9.**

| Feature | Props | State |
|---|---|---|
| Definition | Data passed to component | Data managed inside component |
| Mutability | Immutable | Mutable |
| Controlled By | Parent component | Component itself |
| Purpose | Communication | Component behavior |

**Hints:**
Props = external data, State = internal data.

---

## Q10. What is a Functional Component?

**Ans10.**

A **Functional Component** is a simple JavaScript function that returns JSX.

Example:

```
function Greeting() {
  return <h1>Hello React</h1>;
}
```

With Hooks:

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
}
```

**Advantages:**

➢ Simpler and cleaner syntax
➢ Easier to test
➢ Supports hooks

**Hints:**
Modern React development mainly uses **functional components instead of class components**.

## Q11. What is a Class Component in React?

**Ans11.**
A **Class Component** is a React component written using ES6 class syntax. It extends `React.Component` and must contain a `render()` method that returns JSX.

Example:

```
import React, { Component } from "react";

class Welcome extends Component {
  render() {
    return <h1>Hello React</h1>;
  }
}
```

**Key Points:**

➢ Uses ES6 classes
➢ Can use lifecycle methods
➢ Maintains state using `this.state`

**Hints:**
Before React Hooks, class components were the primary way to manage state and lifecycle.

---

## Q12. What are Lifecycle Methods in React?

**Ans12.**

Lifecycle methods are special methods in class components that allow developers to execute code at specific stages of a component's life.

**Three main phases:**

➢ **Mounting:** Component is created and inserted into the DOM
➢ **Updating:** Component re-renders due to state or props changes
➢ **Unmounting:** Component is removed from the DOM

**Examples:**

- `componentDidMount()`
- `componentDidUpdate()`
- `componentWillUnmount()`

**Hints:**
Lifecycle methods help manage API calls, timers, and cleanup operations.

---

## Q13. What is `componentDidMount()`?

**Ans13.**

`componentDidMount()` is a lifecycle method that runs **after the component is rendered for the first time**.

Example:

```
componentDidMount() {
  console.log("Component mounted");
}
```

**Common Uses:**

➢ Fetching API data
➢ Starting timers
➢ DOM manipulation

**Hints:**
This method is similar to the `useEffect()` hook in functional components.

---

## Q14. What is `componentDidUpdate()`?

**Ans14.**

`componentDidUpdate()` is a lifecycle method that runs **after the component updates due to state or props changes**.

Example:

```
componentDidUpdate(prevProps, prevState) {
  if (prevState.count !== this.state.count) {
    console.log("Count changed");
  }
}
```

**Key Points:**

➢ Runs after re-render
➢ Used for side effects after updates
➢ Must compare previous and current values

**Hints:**
Used when UI changes based on state updates.

---

## Q15. What is `componentWillUnmount()`?

**Ans15.**

`componentWillUnmount()` is called **just before a component is removed from the DOM**.

Example:

```
componentWillUnmount() {
  console.log("Component removed");
}
```

**Common Uses:**

➢ Clearing timers
➢ Canceling API requests
➢ Removing event listeners

**Hints:**
It helps prevent memory leaks.

---

## Q16. What is the `render()` method in React?

**Ans16.**

The `render()` method is required in class components and is responsible for returning the JSX that will be displayed on the screen.

Example:

```
render() {
  return <h1>Hello React</h1>;
}
```

**Key Points:**

➢ Must return JSX
➢ Should be pure (no side effects)
➢ Runs whenever state or props change

**Hints:**
React updates the UI based on what the render method returns.

---

## Q17. What are React Hooks?

**Ans17.**

Hooks are functions that allow **functional components to use state and lifecycle features**.

Hooks were introduced in React 16.8.

Common Hooks:

➢ `useState()` – manages state
➢ `useEffect()` – handles side effects
➢ `useContext()` – accesses context

**Example:**

```
const [count, setCount] = useState(0);
```

**Hints:**
Hooks eliminate the need for class components.

---

## Q18. What is the `useState` Hook?

**Ans18.**

`useState` is a React Hook used to add state to functional components.

Example:

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
}
```

**Explanation:**

➢ `count` → state variable
➢ `setCount` → function to update state

**Key Points:**

➢ Updates trigger re-render
➢ Can store any data type
➢ Multiple states allowed

---

## Q19. What is the `useEffect` Hook?

**Ans19.**

`useEffect` is used to perform **side effects** in functional components.

Side effects include:

➢ API calls
➢ DOM updates
➢ Timers
➢ Event listeners

Example:

```
import { useEffect } from "react";

useEffect(() => {
  console.log("Component mounted");
}, []);
```

**Key Points:**

➢ Runs after render
➢ Dependency array controls execution
➢ Can replace lifecycle methods

---

## Q20. What is the Dependency Array in `useEffect`?

**Ans20.**

The dependency array controls **when the `useEffect` function runs**.

Example:

```
useEffect(() => {
  console.log("Runs once");
}, []);
```

**Cases:**

➢ `[]` → Runs only once (on mount)
➢ `[value]` → Runs when value changes
➢ No array → Runs on every render

**Hints:**
The dependency array prevents unnecessary re-renders and improves performance.

## Q21. How do you handle events in React?

**Ans21.**

React uses **camelCase syntax** for events and passes a **function reference** instead of a string.

Example:

```
function Button() {
  function handleClick() {
    console.log("Button clicked");
  }

  return <button onClick={handleClick}>Click Me</button>;
}
```

**Key Points:**

➢ Use camelCase (`onClick`, `onChange`)
➢ Pass function reference, not a string
➢ Can access event object like `event.target`

**Hints:**
React normalizes events using **SyntheticEvent** for cross-browser compatibility.

---

## Q22. What is `SyntheticEvent` in React?

**Ans22.**

`SyntheticEvent` is a **cross-browser wrapper** around the native browser event. It ensures consistency across different browsers.

**Example:**

```
function handleChange(event) {
  console.log(event.target.value);
}
```

**Key Points:**

➢ Normalizes native events
➢ Works in all browsers
➢ Methods like `preventDefault()` are available

---

## Q23. How do you perform conditional rendering in React?

**Ans23.**

Conditional rendering allows you to render elements **based on conditions** using JavaScript expressions.

**Example 1: Using `if`**

```
function Greeting({ isLoggedIn }) {
  if (isLoggedIn) {
    return <h1>Welcome User</h1>;
  } else {
    return <h1>Please Login</h1>;
  }
}
```

**Example 2: Using Ternary Operator**

```
<h1>{isLoggedIn ? "Welcome" : "Please Login"}</h1>
```

**Hints:**
Conditional rendering is commonly used for authentication, UI updates, or feature toggling.

---

## Q24. How do you render lists in React?

**Ans24.**

Use the `map()` method to render arrays of items into components.

Example:

```
const names = ["Pushkar", "Ravi", "Anita"];

function NameList() {
  return (
    <ul>
      {names.map((name, index) => (
        <li key={index}>{name}</li>
      ))}
    </ul>
  );
}
```

**Key Points:**

➢ Always include a **unique `key`**
➢ Helps React identify elements for efficient updates
➢ Can map over objects or arrays

**Hints:**
Keys prevent unnecessary re-renders and improve performance.

---

## Q25. What is the purpose of keys in React lists?

**Ans25.**

Keys are **unique identifiers** for elements in a list.

**Purpose:**

➢ Helps React **track items efficiently**
➢ Minimizes unnecessary re-rendering
➢ Should be **stable and unique**, not using array index if items can reorder

---

## Q26. How do you handle forms in React?

**Ans26.**

Forms in React are handled using **controlled components**, where form data is stored in the component's state.

Example:

```
function MyForm() {
  const [name, setName] = useState("");

  function handleChange(event) {
    setName(event.target.value);
  }

  return (
    <form>
      <input type="text" value={name} onChange={handleChange} />
    </form>
  );
}
```

**Key Points:**

➢ Use state to manage input values
➢ `onChange` updates the state
➢ State reflects form input

---

## Q27. What is a Controlled Component?

**Ans27.**

A **controlled component** is a form element whose value is controlled by React state.

**Example:**

```
<input type="text" value={name} onChange={handleChange} />
```

**Key Points:**

➢ React state is the single source of truth
➢ Makes form validation and data handling easier
➢ All input changes must update state

---

## Q28. What is an Uncontrolled Component?

**Ans28.**

An **uncontrolled component** stores its own state internally rather than in React state.

**Example:**

```
<input type="text" ref={inputRef} />
```

**Key Points:**

➢ Use `ref` to access value
➢ Less control, suitable for simple forms
➢ Avoid for complex forms or validation

---

## Q29. How do you handle multiple input fields in React?

**Ans29.**

Use a **single state object** for multiple inputs and update it dynamically.

Example:

```
const [formData, setFormData] = useState({ name: "", email: "" });

function handleChange(event) {
  setFormData({
    ...formData,
    [event.target.name]: event.target.value,
  });
}
```

**Hints:**
Use `[event.target.name]` to update the correct field dynamically.

---

## Q30. How do you prevent the default form submission in React?

**Ans30.**

Use the `preventDefault()` method on the event object.

Example:

```
function handleSubmit(event) {
  event.preventDefault();
  console.log("Form submitted");
}
```

**Hints:**
Prevents page reload, allowing you to handle form data with React.

## Q31. What is React Router?

**Ans31.**

React Router is a library used to implement **routing in React applications**. It allows navigation between different components without refreshing the page, enabling **Single Page Application (SPA)** behavior.

**Key Points:**

➢ Maintains browser history
➢ Renders components based on URL
➢ Supports nested and dynamic routes

**Example:**

```
import { BrowserRouter as Router, Route, Routes } from "react-router-dom";
import Home from "./Home";
import About from "./About";

function App() {
  return (
    <Router>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </Router>
  );
}
```

---

## Q32. What is the difference between BrowserRouter and HashRouter?

**Ans32.**

| Feature | BrowserRouter | HashRouter |
|---|---|---|
| URL Format | `/about` | `/#/about` |

| Feature | BrowserRouter | HashRouter |
|---|---|---|
| Uses | HTML5 history API | Hash portion of URL |
| Server Setup | Requires server configuration | Works without server setup |
| Use Case | Modern apps with server | Static hosting without server support |

**Hints:**
BrowserRouter is preferred for production apps with proper server setup.

---

## Q33. What is a Link in React Router?

**Ans33.**

`<Link>` is a component used to **navigate between routes** without reloading the page.

Example:

```
import { Link } from "react-router-dom";

<Link to="/about">Go to About</Link>
```

**Key Points:**

➢ Replaces `<a>` tag in SPAs
➢ Prevents full page reloads
➢ Works with BrowserRouter or HashRouter

---

## Q34. What is the difference between Link and NavLink?

**Ans34.**

| Feature | Link | NavLink |
|---|---|---|
| Purpose | Navigate between routes | Navigate + add active class |
| Styling | No automatic styling | Adds `active` class when route matches |
| Use Case | Simple links | Navigation menus with active highlighting |

Example of NavLink:

```
<NavLink to="/about" activeClassName="active">About</NavLink>
```

---

## Q35. What is React Context API?

**Ans35.**

React Context API allows **sharing data across components without passing props manually** through every level.

**Example:**

```
import { createContext, useContext } from "react";

const ThemeContext = createContext("light");

function Component() {
  const theme = useContext(ThemeContext);
  return <div>{theme}</div>;
}
```

**Key Points:**

➢ Solves **prop drilling**
➢ Works with Provider and Consumer
➢ Best for global state like themes, user info

---

## Q36. How do you create a Context?

**Ans36.**

1. **Create context:**

```
const MyContext = createContext(defaultValue);
```

2. **Provide value:**

```
<MyContext.Provider value={value}>
  <ChildComponent />
</MyContext.Provider>
```

3. **Consume value:**

- Using `useContext` in functional component:

```
const value = useContext(MyContext);
```

---

## Q37. What is Redux in React?

**Ans37.**

Redux is a **state management library** often used with React. It helps manage the application state in a **centralized store**.

**Key Points:**

➢ Predictable state management
➢ Central store contains the app state
➢ Actions and reducers update the store

**Basic Flow:**

```
Action → Reducer → Store → UI
```

---

## Q38. What is the difference between Context API and Redux?

**Ans38.**

| Feature | Context API | Redux |
|---|---|---|
| Scope | Lightweight, simple apps | Complex apps with large state |
| Store | Built-in | External library |
| Dev Tools | Limited | Advanced DevTools |
| Middleware | Not built-in | Supports middleware like thunk or saga |

**Hints:**
Context is better for small, simple apps; Redux for complex, large-scale state management.

---

## Q39. How do you update Context value dynamically?

**Ans39.**

Pass a **state and setter function** in the Provider.

Example:

```
const [theme, setTheme] = useState("light");

<ThemeContext.Provider value={{ theme, setTheme }}>
  <Child />
</ThemeContext.Provider>
```

Child Component:

```
const { theme, setTheme } = useContext(ThemeContext);
setTheme("dark");
```

---

## Q40. What is the difference between local state and global state?

**Ans40.**

| Feature | Local State | Global State |
|---|---|---|
| Scope | Component-level | Application-level |
| Managed By | `useState` or `this.state` | Context API, Redux, MobX |
| Example | Input value, toggle | User authentication, theme, cart items |
| Sharing | Hard to share between components | Easily shared |

**Hints:**
Local state is simple, while global state is needed for shared data across multiple components.

## Q41. What are some common performance issues in React?

**Ans41.**

Common issues include:

➢ **Unnecessary re-renders** of components
➢ **Large component trees**
➢ **Inefficient list rendering** without keys
➢ **Heavy computations** inside render
➢ **Improper use of state or props** causing cascading updates

**Hints:**
Performance optimization focuses on minimizing re-renders and managing state efficiently.

---

## Q42. How do you prevent unnecessary re-renders?

**Ans42.**

Techniques:

1. **React.memo()** – Memoizes functional components to avoid re-rendering if props didn't change.

```
const MyComponent = React.memo(function Component({ value }) {
  return <div>{value}</div>;
});
```

2. **useCallback()** – Memoizes functions passed as props.
3. **useMemo()** – Memoizes computed values to avoid recalculation.
4. **PureComponent** – For class components, automatically implements shallow comparison.

---

## Q43. What is React.lazy()?

**Ans43.**

`React.lazy()` allows **lazy loading of components**, i.e., loading them only when needed.

Example:

```
import React, { Suspense, lazy } from "react";

const MyComponent = lazy(() => import("./MyComponent"));

function App() {
  return (
    <Suspense fallback={<div>Loading...</div>}>
      <MyComponent />
    </Suspense>
  );
}
```

**Hints:**
Reduces initial bundle size and improves performance.

---

## Q44. What is Suspense in React?

**Ans44.**

`Suspense` is used to **wrap lazy-loaded components** and provide a fallback UI until the component is loaded.

Example:

```
<Suspense fallback={<div>Loading...</div>}>
  <LazyComponent />
</Suspense>
```

**Key Points:**

➢ Works with `React.lazy()`
➢ Can also be used for data fetching in future React versions
➢ Improves user experience by showing loading UI

---

## Q45. What is code splitting in React?

**Ans45.**

Code splitting is a technique to **split JavaScript bundles** into smaller chunks, loading only what is needed for a particular page or component.

**Benefits:**

➢ Reduces initial load time
➢ Improves performance
➢ Works with `React.lazy()` and dynamic `import()`

**Example:**

```
const Home = lazy(() => import("./Home"));
```

---

## Q46. What is the difference between useMemo and useCallback?

**Ans46.**

| Feature | useMemo | useCallback |
|---|---|---|
| Purpose | Memoizes **values** | Memoizes **functions** |
| Usage | Avoids expensive calculations | Avoids function re-creation |
| Returns | Cached value | Cached function |

Example:

```
// useMemo
const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a, b]);

// useCallback
const memoizedFunction = useCallback(() => doSomething(a), [a]);
```

---

## Q47. What is PureComponent?

**Ans47.**

`PureComponent` is a **class component** that implements **shouldComponentUpdate()** with shallow prop and state comparison.

**Example:**

```
class MyComponent extends React.PureComponent {
  render() {
    return <div>{this.props.value}</div>;
  }
}
```

**Hints:**
Reduces unnecessary re-renders for class components.

---

## Q48. How can you optimize lists in React?

**Ans48.**

Techniques:

➢ Always use **unique keys**
➢ Use **React.memo()** for list items
➢ Avoid inline functions inside `map()`
➢ Pagination or virtualization (e.g., `react-window`) for large lists

**Hints:**
Optimizing lists improves performance when rendering many elements.

---

## Q49. What is reconciliation in React?

**Ans49.**

Reconciliation is the **process React uses to update the DOM** when state or props change.

**Steps:**

1. Create a new Virtual DOM
2. Compare with previous Virtual DOM (diffing)
3. Apply minimal changes to the real DOM

**Hints:**
Reconciliation ensures efficient updates with minimal DOM operations.

---

## Q50. What are best practices for React performance?

**Ans50.**

➢ Split code using `React.lazy()`
➢ Use memoization (`React.memo`, `useMemo`, `useCallback`)
➢ Keep component state minimal and local
➢ Avoid inline functions and unnecessary renders
➢ Use keys for list rendering
➢ Optimize images and assets
➢ Use React DevTools to identify performance bottlenecks

**Hints:**
Performance tuning ensures faster, scalable, and smoother apps.

## Q51. What is a React Portal?

**Ans51.**

A **React Portal** allows rendering a component **outside the parent DOM hierarchy**. It's useful for modals, tooltips, or overlays.

**Example:**

```
import ReactDOM from "react-dom";

function Modal({ children }) {
  return ReactDOM.createPortal(
    <div className="modal">{children}</div>,
    document.getElementById("modal-root")
  );
}
```

**Key Points:**

➢ Keeps component logic inside React but renders elsewhere in DOM
➢ Useful for overlay UI elements
➢ Parent component state can still control it

---

## Q52. What are Error Boundaries in React?

**Ans52.**

Error boundaries are **React components that catch JavaScript errors in their child component tree** and display a fallback UI instead of crashing the entire app.

**Example:**

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError(error) {
    return { hasError: true };
  }

  render() {
    if (this.state.hasError) {
      return <h1>Something went wrong.</h1>;
    }
    return this.props.children;
  }
}
```

**Hints:**
Functional components cannot be error boundaries (as of now).

---

## Q53. What is a Higher-Order Component (HOC)?

**Ans53.**

A **Higher-Order Component** is a function that **takes a component and returns a new enhanced component**.

**Example:**

```
function withLogger(WrappedComponent) {
  return function (props) {
    console.log("Rendering", WrappedComponent.name);
    return <WrappedComponent {...props} />;
  };
}
```

**Key Points:**

➢ Used for code reuse and logic abstraction
➢ Can enhance multiple components
➢ Often used for authentication, logging, or theme handling

---

## Q54. What are React Fragments?

**Ans54.**

React Fragments allow grouping **multiple elements without adding extra nodes** to the DOM.

**Example:**

```
function List() {
  return (
    <>
      <li>Item 1</li>
      <li>Item 2</li>
    </>
  );
}
```

**Hints:**
Fragments reduce unnecessary wrapper elements like `<div>`.

---

## Q55. What is the difference between HOC and Render Props?

**Ans55.**

| Feature | HOC | Render Props |
|---|---|---|
| Definition | Function that returns a component | Component uses a function as child |
| Use Case | Code reuse, logic abstraction | Dynamic rendering, flexible UI |
| Syntax | `withFeature(Component)` | `<Component>{data => <UI />}</Component>` |

---

## Q56. What is the purpose of `getDerivedStateFromError`?

**Ans56.**

`getDerivedStateFromError` is a **static lifecycle method** used in error boundaries to **update state when a child component throws an error**.

**Example:**

```
static getDerivedStateFromError(error) {
  return { hasError: true };
}
```

**Hints:**
It triggers a re-render to show fallback UI.

---

## Q57. How is `componentDidCatch` different from `getDerivedStateFromError`?

**Ans57.**

| Feature | getDerivedStateFromError | componentDidCatch |
|---|---|---|
| Purpose | Update state to show fallback UI | Log error information |
| Called | Before render | After render |
| Parameters | error | error, info |

**Hints:**
Use both together in error boundaries: one for UI, one for logging.

---

## Q58. Can functional components use HOCs?

**Ans58.**

Yes, **functional components can be wrapped in HOCs** to enhance behavior.

Example:

```
function Hello() {
  return <h1>Hello</h1>;
}

const EnhancedHello = withLogger(Hello);
```

**Hints:**
HOCs work with both class and functional components.

---

## Q59. What are the advantages of React Portals?

**Ans59.**

➢ **Avoid CSS conflicts** by rendering outside parent
➢ **Maintain component state** while rendering elsewhere
➢ **Useful for overlays** like modals, tooltips, and dropdowns
➢ **Easy to manage** events inside the portal

---

## Q60. What are common use cases for React Fragments?

**Ans60.**

➢ Return multiple elements from a component without extra DOM nodes
➢ Optimize performance by reducing unnecessary `<div>` wrappers
➢ Useful in lists, tables, and grouping multiple child components

SOFTPRO INDIA